Chapter 8

# A Case Study on Ontology-Based Evaluation and Re-Design

In this chapter, we conduct two complementary case studies of the techniques developed in this thesis.

Firstly, in order to demonstrate the language evaluation framework proposed in chapter 2, we systematically analyze and re-design the Unified Modeling Language (UML) as a conceptual modeling language. To achieve this objective, we employ the *foundational ontology* proposed throughout chapters 4 to 7 of this thesis as a basis for the evaluation of the current UML 2.0 metamodel, and for the definition of the real-world semantics of the ontologically well-founded version of UML that results from this re-designing process. This process is conducted in three complementary sections: section 8.1 uses the results mainly from chapter 4 to address the UML metamodel elements of *class* and *class taxonomies*; section 8.2 uses the results mainly from chapter 6, taking a broader view on UML classifiers (which include, besides classes, also associations, datatypes and interfaces), but also analyzing the UML *Property* metaclass (e.g., attributes and association ends); Finally, section 8.3 uses the results mainly from chapter 5 to address the representation of meronymic relations (e.g., composition and aggregation) in the UML metamodel. We employ the same structure in each of these three subsections: each subsection contains two parts; in the first part we always present a relevant fragment of the UML metamodel; in a second part we do three things, namely, (a) analyze this fragment in terms of a suitable part of our foundational ontology; (b) propose a revised version of the UML metamodel in conformance with the foundation ontology; (c) propose a modelling profile that implements the revised metamodel.

Secondly, once the language has been re-designed, in section 8.5 we demonstrate its adequacy in tackling some recurrent representation problems as well as some problems of semantic interoperability in the

modeling and integration of *lightweight ontologies* to be used for context-aware service platforms.

Section 8.5 presents some final considerations for this chapter.

## 8.1 Classes and Generalization

### 8.1.1 The UML Metamodel

In this section we briefly present the UML metamodel, focusing on static elements and, in particular, those modeling primitives used in the construction of UML class diagrams. From now on, we refer to the *UML 2.0 Superstructure Specification* (OMG, 2003c) when quoting in italics the definition of elements of the UML metamodel.

Figure 8.1 below depicts a fragment of the UML metamodel featuring the metaclasses which are more relevant for the purpose of this discussion. In particular, we focus our discussion on those meta-constructs highlighted in this figure, namely, *Classifier*, *Class*, *Generalization* and *GeneralizationSet*.

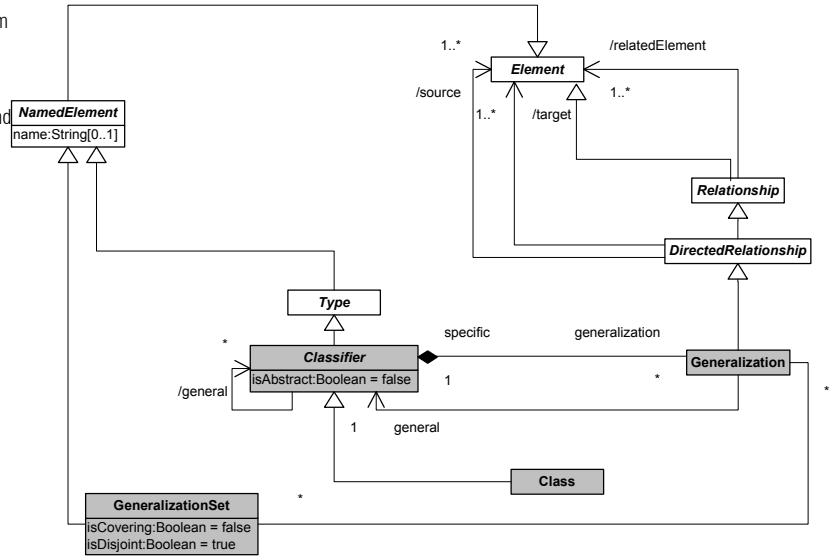



*Figure 8-1* Excerpt from the UML metamodel featuring the metaclasses *Classifier*, *Class*, *Generalization* and *GeneralizationSet*

In the UML superstructure specification, a classifier is defined as follows: *"A classifier is a classification of instances according to their features. [It] may participate in generalization relationships with other Classifiers [, in which case] an instance of a specific Classifier is also an (indirect) instance of each of the general Classifiers. Any constraint applying to instances of the general classifier also applies to*

*instances of the specific classifier. A Classifier defines a type. Type conformance between generalizable Classifiers is defined so that a Classifier conforms to itself and to all of its ancestors in the generalization hierarchy. [It] can specify a generalization hierarchy by referencing its general classifiers. If [the classifier's attribute isAbstract =] true, the Classifier does not provide a complete declaration and can typically not be instantiated. An abstract classifier is intended to be used by other classifiers, e.g. as the target of general metarelationships or generalization relationships. [The] default value is false."*

A classifier in UML is a general abstract metaclass that subsumes other metaclasses such as *Class*, *Association*, *Interface* and *Datatype*. Its main purpose is to describe the general aspects of property (attribute) description and generalization hierarchies (taxonomies) for its subclasses. As recognized in the UML specification, among its different subtypes *"class is the most widely used classifier"*. A class is a type of classifier that *"describes a set of Objects sharing a collection of Features, including Operations, Attributes and Methods, that are common to the set of Objects… [Its purpose] is to specify a classification of objects and to specify the features that characterize the structure and behavior of those objects"*.

In UML, a generalization is shown as a line with a hollow triangle as an arrowhead between the symbols representing the involved classifiers. The arrowhead points to the symbol representing the general classifier. A generalization relationship may be connected to a *GeneralizationSet*. This can be depicted graphically in one the following ways:

1. By placing the name of the GeneralizationSet explicitly in an adjacent position to that of the generalization symbol. Therefore, all Generalization relationships with the same GeneralizationSet name are part of the same GeneralizationSet (figure 8.2.a);

2. By drawing two or more lines (ends of the generalization line symbol) that converge to a single arrowhead. This symbolizes that all the specific classifiers belong to the same GeneralizationSet (figure 8.2.b);

3. By drawing a dashed line across those lines with separate arrowheads that are meant to be part of the same set. This dashed line designates the GeneralizationSet (figure 8.2.c).

In cases (b) and (c), the representation of the GeneralizationSet name is optional.

A GeneralizationSet defines a particular set of Generalization relationships that describe the way in which a specific Classifier (or superclass) may be partitioned. For example, in figures 8.2.a, 8.2.b and 8.2.c below, a GeneralizationSet named *gender* defines the partitioning of class Person in the two subclasses: MalePerson and FemalePerson.

However, in the UML specification, the term *partition* is not used necessarily in the mathematical sense. This is only the case if both attributes *isCovering* and *isDisjoint* of a GeneralizationSet have the *true* value. In the case of the aforementioned example, if (*isCovering* = *true*) then every instance of Person must be either an instance of MalePerson or FemalePerson, i.e., in every world the extensions of the subclasses that are part of the generalization set exhaust the extension of their superclass. If (*isDisjoint* = *true*) then there is no instance of Person who can be both and instance of MalePerson and of FemalePerson, i.e., within a world, the extensions of the subclasses that are part of the generalization set are necessarily mutually disjoint. The default interpretation in UML is that (*isDisjoint* = *true*) but (*isCovering* = *false*).

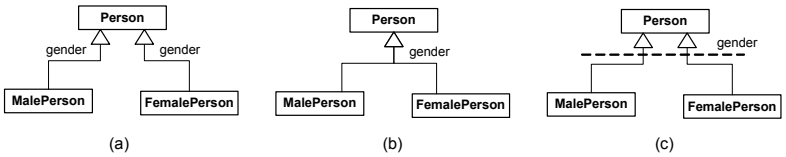



*Figure 8-2* Alternative representations for a GeneralizationSet

### 8.1.2 Ontological Interpretation and Re-Design

In this section we focus in a special sense of the UML metaclass *Class*. By class hereby we mean the notion of a first-order class, as opposed to *powertypes*, and one whose instances are single objects, as opposed to *association classes*, whose instances are tuples of objects.

In this sense, the ontological interpretation of a UML Class is that of a *monadic universal* (see figure 8.3). Class diagrams are intended to represent the static structure of a domain. For this reason, we take here that classes should always be interpreted as representing *endurant universals*. In addition, since substantials are prior to Moments from an identification point of view, i.e., the latter are *identificationally dependent* on the former (Schneider, 2003b), it is typically the case that most classes in a class diagram should be interpreted as substantial universals. In fact, we prescribe that a non-stereotyped class in a class diagram should be interpreted as a substantial universal.

Classes are therefore the representation of general terms in a description of a domain. A general requirement in conceptual modeling languages is that the represented instances must have a definite identity (Borgida, 1990). For this reason, classes in a class diagram representing substantial universals should always be interpreted as *Object universals*.

In chapter 4 of this thesis, we present a theory that proposes a number of ontological distinctions that refine the category of *Object Universal*. One of the sorts of object universal is a substance sortal, or a *Kind*. As discussed in

depth in chapter 5 of this thesis, within the category of *kinds*, we can make three further types distinctions, based on the ontological categories of their instances. These distinctions are *quantity* kind, *collective* kind and *functional complex* kind. Functional complex kinds are certainly the most commonly represented kinds in conceptual specifications. For this reason, for now on, we simply write *kind* when referring to a functional complex kind. These ontological distinctions are depicted in figure 8.3 below.

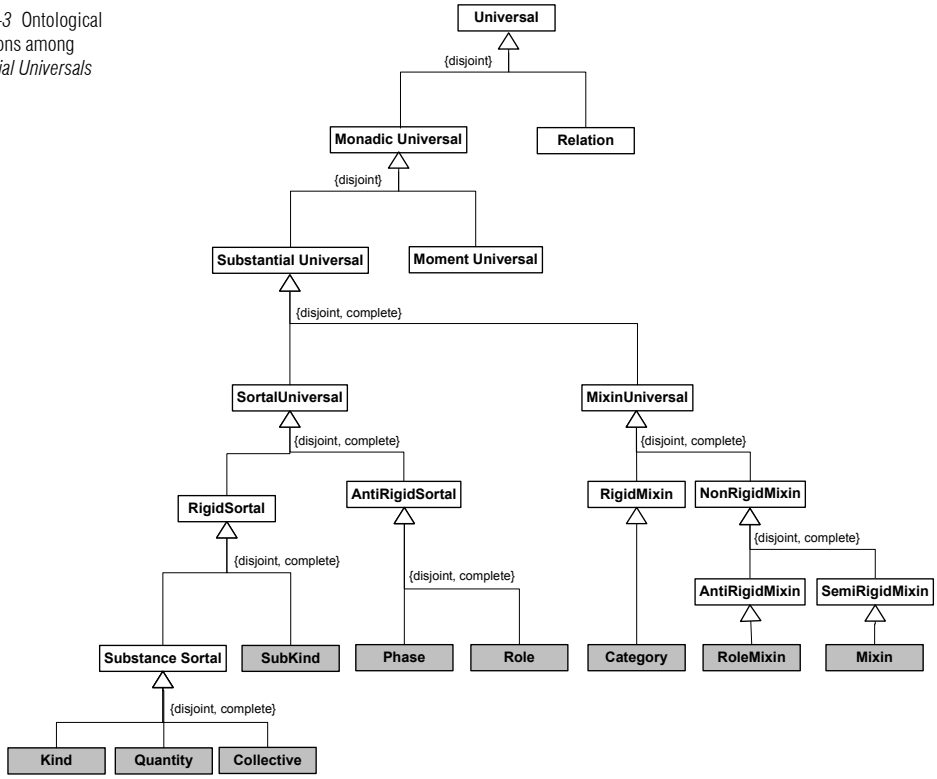
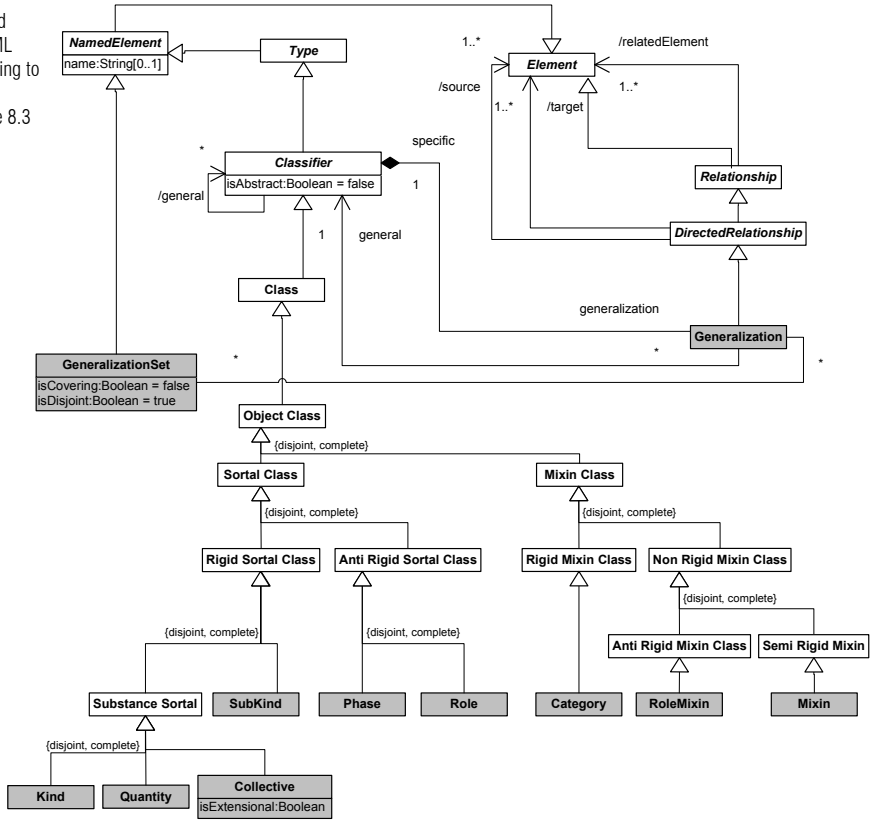



*Figure 8-3* Ontological Distinctions among *Substantial Universals*

If we make a representation mapping from the ontology of figure 8.3 to the UML metamodel, we can map the category of *Monadic Universal* to the UML element of a *Class*. However, by carrying on this process, we realize that in UML there are no modeling constructs that represent the ontological categories specializing *Object Universal* in this figure. In other words, there are ontological concepts prescribed by our reference ontology that are not represented by any modeling construct in the language. This amounts to a

case of ***ontological incompleteness*** at the modeling language level (see section 2.3.4).

*Figure 8-4* Revised fragment of the UML metamodel according to the ontological categories of figure 8.3




In order to remedy this problem, we propose a first extension to UML that represents finer-grained distinctions between different types of classes. We, thus, define extensions of the UML metaclass *Class* to represents each of the leaf ontological categories specializing *substantial universal* in figure 8.3. The extended UML metamodel must also include a number of constraints (derived from the postulates of the theories presented in chapters 4 and 5) that restrict the ways the introduced elements can be related. The goal is to have a metamodel such that all syntactically correct specifications according to this metamodel have logical models that are *intended world structures* of the conceptualizations they are supposed to represent (see section 3.4.1).

In figure 8.4 above we present an extension of the UML meta-model in order to represent the ontological concepts discussed in this section. The

concrete metaclasses in this figure (highlighted in grey) represent the leaf ontological distinctions among the category of *Object Universals* prescribed by our theory.

In the sequel, we define a *profile* that implements the metaclasses of this (extended) UML metamodel, as well as their interrelationships and contraints. By using this profile, the concrete *Object classes* in figure 8.4 are represented in conceptual models as stereotyped classes representing each of the considered ontological distinctions. Likewise, the admissible relations between these ontological categories, derived from the postulates of our theory, are represented in the profile as syntactical constraints governing the admissible relations between the corresponding stereotyped classes. This modeling profile is presented in table 8.1 below.

*Table 8-1* Ontologically Well-Founded UML modelling profile according to the ontological categories of figure 8.3

| Metaclass | Description |
|---|---|
| Substance Sortal | *Substance Sortal* is an abstract metaclass that represents the general properties of all *substance sortals*, i.e., rigid, relationally independent object universals that supply a principle of identity for their instances. Substance Sortal has no concrete syntax. Thus, symbolic representations are defined by each of its concrete subclasses. |
| **Constraints** | |
| 1. Every substantial object represented in a conceptual model using this profile must be an instance of a substance sortal, directly or indirectly. This means that every concrete element of this profile used in a class diagram (isAbstract = false) must include in its general collection one class stereotyped as either «kind», «quantity» or «collective». This constraint is a refinement of postulate 4.1 in chapter 4;<br><br>2. A substantial object represented in a conceptual model using this profile cannot be an instance of more than one ultimate substance sortal. This means that any stereotyped class in this profile used in a class diagram must not include in its general collection more than one substance sortal class. Moreover, a substance sortal must also not include another substance sortal nor a « subkind » in its general collection, i.e., a substance sortal cannot have as a supertype a member of {«kind», «subkind», «quantity», «collective»}. This constraint is a refinement of postulate 4.2 in chapter 4;<br><br>3. A Class representing a rigid substantial universal cannot be a subclass of a Class representing an anti-rigid universal. Thus, a substance sortal cannot have as a supertype (must not include in its general collection) a member of {«phase», «role», «roleMixin»}. This constraint is a result of postulate 4.3 in chapter 4. | |
| **Stereotype** | **Description** |
| «kind»<br>**A** | A «kind» represents a *substance sortal* whose instances are *functional complexes*. Examples include instances of Natural Kinds (such as Person, Dog, Tree) and of artifacts (Chair, Car, Television). |

| Stereotype | Description |
|---|---|
| «quantity» **A** | A «quantity» represents a substance sortal whose instances are *quantities*. Examples are those stuff universals that are typically referred in natural language by mass general terms (e.g., Gold, Water, Sand, Clay). |

| Stereotype | Description |
|---|---|
| «collective» **A** | A «collective» represents a substance sortal whose instances are *collectives*, i.e., they are collections of complexes that have a uniform structure. Examples include a deck of cards, a forest, a group of people, a pile of bricks. Collectives can typically relate to complexes via a constitution relation. For example, a pile of bricks that *constitutes* a wall, a group of people that *constitutes* a football team. In this case, the collectives typically have an extensional principle of identity, in contrast to the complexes they constitute. For instance, The Beatles was in a given world w constituted by the collective {John, Paul, George, Pete} and in another world w' constituted by the collective {John, Paul, George, Ringo}. The replacement of Pete Best by Ringo Star does not alter the identity of the *band*, but creates a numerically different *group of people*. |

| Constraints |
|---|
| 1. A collective can be extensional. In this case the meta-attribute *isExtensional* defined in figure 8.4 is equal to *True*. This means that all its parts are essential and the change (or destruction) of any of its parts terminates the existence of the collective. We use the symbol {extensional} to represent an extensional collective. |

| Stereotype | Description |
|---|---|
| «subkind» **A**<br>**A** | A «subkind» is a rigid, relationally independent restriction of a substance sortal that carries the principle of identity supplied by it. An example could be the subkind MalePerson of the kind Person. In general, the stereotype «subkind» can be omitted in conceptual models without loss of clarity. |

| Constraints |
|---|
| 1. Following postulate 4.3, a «subkind» cannot have as a supertype (must not include in its general collection) a member of {«phase», «role», «roleMixin»}. |

| Stereotype | Description |
|---|---|
| «phase» **A** | A «phase» represents the phased-sortals *phase,* i.e. *anti-rigid* and *relationally independent* universals defined as part of a partition of a substance sortal. For instance, ⟨Catterpillar, Butterfly⟩ partitions the kind Lepdopterum. |

**Constraints**

1. Phases are anti-rigid universals and, thus, a «phase» cannot appear in a conceptual model as a supertype of a rigid universal (postulate 4.3);

2. The phases $\{P_1 \ldots P_n\}$ that form a phase-partition of a substance sortal S are represented in a class diagram as a disjoint and complete generalization set. In other words, a GeneralizationSet with (isCovering = true) and (isDisjoint = true) is used in a representation mapping as the representation for the ontological concept of a phase-partition.

| Stereotype | Description |
|---|---|
| «role» **A** | A «role» represents a phased-sortal role, i.e. anti-rigid and relationally dependent universal. For instance, the role student is played by an instance of the kind Person. |

**Constraints**

1. Roles anti-rigid universals and, thus, a «role» cannot appear in a conceptual model as a supertype of a rigid universal (postulate 4.3);

2. Let X be a class stereotyped as «role» and r be an association representing X's restriction condition. Then, $\#X.r \geq 1$. This constraint is elaborated in the next section, after additional elements of the UML metamodel are considered.

| Metaclass | Description |
|---|---|
| Mixin Class | *Mixin Class* is an abstract metaclass that represents the general properties of all *mixins*, i.e., *non-sortals* (or dispersive universals). Mixin Class has no concrete syntax. Thus, symbolic representations are defined by each of its concrete subclasses. |

**Constraints**

1. Following postulate 4.4 we have that a class representing a non-sortal universal cannot be a subclass of a class representing a Sortal. As a consequence of this postulate we have that a mixin class cannot have as a supertype (must not include in its general collection) a member of {«kind», «quantity», «collective», «subkind», «phase», «role»}.

2. Moreover, as a consequence of postulate 4.1, a non-sortal cannot have direct instances. Therefore, a mixin class must always be depicted as an abstract class (isAbstract = true).

| Stereotype | Description |
|---|---|
| «category» ***A*** | A «category» represents a rigid and relationally independent *mixin*, i.e., a dispersive universal that aggregates essential properties which are common to different substance sortals. For example, the category RationalEntity as a generalization of Person and IntelligentAgent. |

| Constraints |
|---|
| 1. As a consequence of this postulate and of postulate 4.3, we have that a «category» cannot have a «roleMixin» as a supertype. In other words, together with condition 1 for all mixins we have that a «category» can only be subsumed by another «category» or a «mixin». |

| Stereotype | Description |
|---|---|
| «roleMixin» **A** | A «roleMixin» represents an anti-rigid and externally dependent *non-sortal*, i.e., a dispersive universal that aggregates properties which are common to different roles. In includes formal roles such as *whole* and *part*, and *initiation* and *responder*. |

| Constraints |
|---|
| 1. Let X be a class stereotyped as «roleMixin» and r be an association representing X's restriction condition. Then, #X.r ≥ 1. The latter constraint is elaborated in the next section, after additional elements of the UML metamodel are introduced. |

| Stereotype | Description |
|---|---|
| «mixin» **A** | A «mixin» represents properties which are essential to some of its instances and accidental to others (semi-rigidity). An example is the mixin Seatable, which represents a property that can be considered essential to the kinds Chair and Stool, but accidental to Crate, Paper Box or Rock. |

| Constraints |
|---|
| 1. Due to postulates 4.3, we have that a «mixin» cannot have a «roleMixin» as a supertype. |

## 8.2 Classifiers and Properties

### 8.2.1 The UML Metamodel

*Classifier* is an abstract metaclass in the UML metamodel aimed at describing some general aspects that are common to all its subclasses, such as the mechanisms for participating in generalization hierarchies and for properties description.

A classifier can posses a number of *features*. A Feature represents some (behavioral or structural) characteristic that is common to all possible instances of its featuring classifiers. A *structural feature* is special type of feature that specializes both *typed element* and *multiplicity element*. As a consequence, it can impose a number of constraints on the range and type

of the values that can be assigned to a given feature of its associated classifiers at instantiation time.

Structural feature is an abstract metaclass. Its most important subclass for the present discussion is *property*. When a property is owned by a *class*, an *interface* or *data type* it represents an *attribute* of that classifier. In this case it relates an instance of the classifier to a value of the type of the attribute. For instance, the property *color* can be defined as an attribute of class *Apple*. This property can have a name (since it is a named element), a type (e.g., the dataType *colorValue* as an enumeration of the colors black, white, blue, yellow, red, green), and a multiplicity (e.g., one-to-one). Therefore, every instance of apple must instantiate exactly one value of type *colorValue* for the property color.

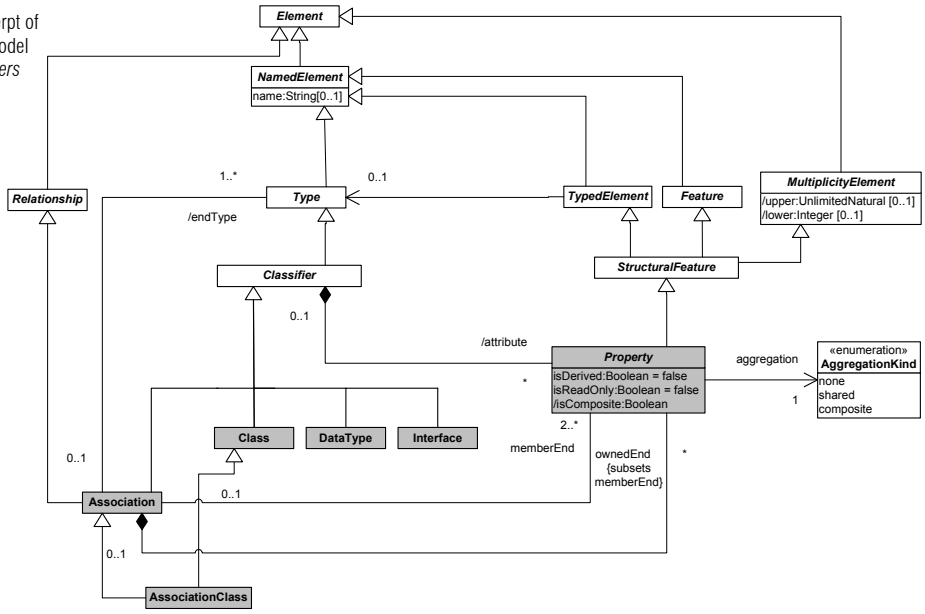



*Figure 8-5* Excerpt of the UML metamodel featuring *classifiers* and *Properties*

A property is qualified according to two meta-attributes, namely, *isReadOnly* and *isDerived*. If the first attribute has the value *true* it means that the value of the property cannot be changed once it has been assigned an initial value. The second attribute indicates whether the property is derived, i.e., whether its value can be computed from other information in the model. For instance, a description of the class *Person* can contain the attribute *data of birth*, and the derivable attribute *age*, which is derived from the former for each given circumstance. The default value of both meta-attributes is *false*.

When related to an association, a property represents an *association end* of that association. An association describes a set of tuples whose values

refer to typed instances. An instance of an association is called a link. Every association must have at least two association ends represented by properties, each of which is connected to the type of that property, which becomes therefore the type of the association end. Associations can be type reflexive, i.e., more than one association end of the same association may have the same type. In figure 8.5, *association* has two different relations with the metaclass property: one via *memberEnd* and another via *ownedEnd*. As represented in this model, the extension of the second one is a subset of that of the first. The first relation represents the association ends that are in fact owned by one of the associated classifiers. These are termed *navigable ends* and they are semantically equivalent to attributes, thus, merely representing an alternative notation. In contrast, in the association connected to ownedEnd, the properties represent type and cardinality restrictions for the possible tuples that instantiate that association. Figure 8.6 below exemplifies a navigable end use as an alternative notation for a classifier's attribute.

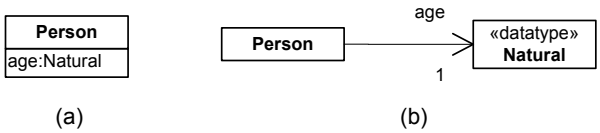



*Figure 8-6 Navigable End* as an alternative notation for representing an *Attribute*

An association may be refined to have its own set of attributes, i.e., attributes that do not belong to any of the connected classifiers but rather to the association itself. Such an association is called an *association class*. An association class is both a kind of association and a kind of a class. As an association it can connect a set of classifiers; as class it can have its own attributes and participate in other associations. Graphically, it is shown in a class diagram as a class symbol attached to the association path by a dashed line. Despite of being graphically distinct, the association path and the association class symbol represent the same underlying model element, which has a single name. An example of an association class is depicted in figure 8.7 below.

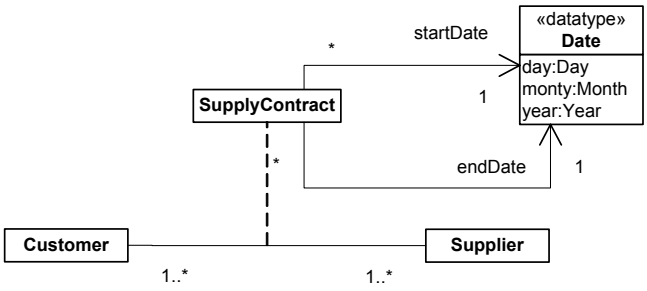



*Figure 8-7* Exemple of an *Association Class* and its owned attributes

In the example above, *Date* is an illustration of a *data type*. According to the UML specification *"a data type is a type whose values have no identity (i.e., they are pure values)…so two occurrences of the same value cannot be differentiated. A DataType defines a kind of classifier in which operations are all pure functions (i.e., they can return data values but they cannot change [them). They are usually] used for specification of the type of an attribute. [Additionally, DataTypes] may have an algebra and operations defined outside of UML, for example, mathematically. [Finally, as any classifier,] a DataType may also contain attributes to support the modeling of structured data types"*.

The remaining type of classifier to be discussed is the metaclass *Interface*. An interface is a declaration of a coherent set of features and obligations. It can be seen as a kind of contract that partitions and characterizes groups of properties that must be fulfilled by any instance of a classifier that implements that interface. The obligations that may be associated with an interface are in the form of various kinds of constraints (such as pre- and postconditions) or protocol specifications, which may impose ordering restrictions on interactions through the interface. Since interfaces are merely declarations, they are non instantiable model elements. That is to say that an interface cannot have direct instances, but only via their implementing classifiers. Figure 8.8 below shows an interface (Seatable), which is realized by two different classes: Crate and Stool. By implementing this interface, these classes commit to provide mechanisms for *"maintaining the information corresponding to the type and multiplicity of the property and facilitate retrieval and modification of that information"*.

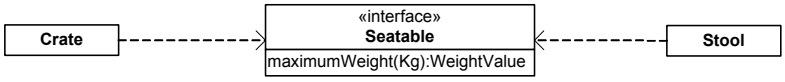



*Figure 8-8* Example of an *Interface* and its implementing classifiers

### 8.2.2 Ontological Interpretation and Re-Design

In section 8.1.2, in an interpretation mapping from the UML metamodel to our foundational ontology, we have mapped the UML class to the ontological category of *monadic universals*. However, we have focused our discussion on the subcategory of *substantial universals*. The other kind of monadic universal prescribed by our theory is *Moment Universal*.

The category of moment universal along with other ontological categories which are relevant for the discussion carried out on this section are depicted in figure 8.9 below.

Moment universals can be differentiated from their substantial counterparts as follows: a moment universal has the characteristic that all its instances inheres in and, thus, is externally existentially dependent on some other individual.

Within the category of moment universals we can use a different criterion to differentiate among its subcategories. An Intrinsic moment universal is a universal which instances are existentially dependent of a single entity (named its bearer). A Relator universal, conversely, is one which instances are existentially dependent of a plurality of entities. Both types of entities are fundamental from an ontological and conceptual point of view. Intrinsic moments constitute a foundation for *attributes* and *comparative formal relations*, but also *weak entities* and *qua individuals*. Qua individuals, in turn, constitute a foundation for *material roles*. Relators are the foundation of *material relations*.

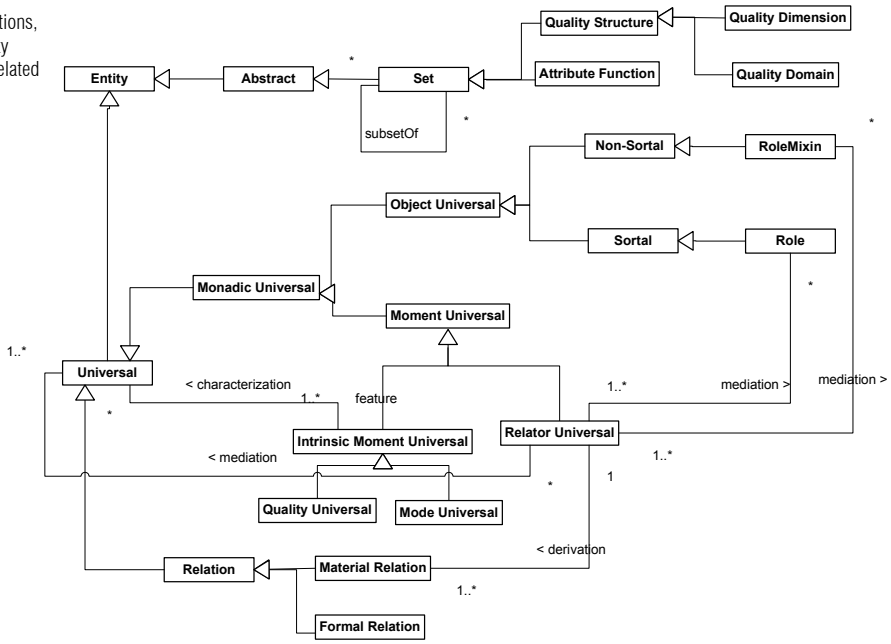



*Figure 8-9* Relations, Moments, Quality Structures and related categories

In a representation mapping from our reference ontology to the UML metamodel, we realize that there are no constructs that represent the ontological categories of *mode universals* and *relator universals* in the UML metamodel. Once more, there are ontological concepts prescribed by our theory that are not represented by any modeling construct in the language, i.e., another case of ***construct incompleteness*** at the modeling language level.

According to postulate 6.1, universals of the domain should be represented in a conceptual model as Classes. We therefore propose an extension of the original UML metamodel of figure 8.5 to incorporate

these entities. Additionally, in a profile implementing this extended metamodel, we include two other stereotyped classes (base class UML class) to represent these entities. Mode universals are represented via a class decorated with the «mode» stereotype. Relator Universals are represented via the stereotype «relator».

Quality universals are typically not represented in a conceptual model explicitly but via *attribute functions* that map each of their instances to points in a quality structure (see discussion on chapter 6). For example, suppose we have the universal Apple (a substantial universal) whose instances exemplify the universal Weight. We say in this case that the quality universal Weight *characterizes* the substantial universal Apple. Thus, for an arbitrary instance *x* of Apple there is a quality *w* (instance of the quality universal Weight) that inheres in *x*. Associated with the universal Weight, and in the context of a given measurement system (e.g., the human perceptual system), there is a quality dimension *weightValue*, which is a set isomorphic to the half line of positive integers obeying the same ordering structure. Quality structures are taken here to be theoretical abstract entities modeled as sets. In this case, we can define an *attribute function* (another abstract theoretical entity) *weight(Kg)*, which maps each instance of apple (and in particular *x*) onto a point in a quality dimension, i.e., its *qualia*.

Following principle 6.2 (section 6.3.2), attribute functions are therefore the ontological interpretation of UML attributes, i.e., UML Properties which are owned by a given classifier. That is, an attribute of a class C representing a universal U is interpreted as an attribute function derived from the *elementary specification* of the universal U.

As any property, a UML attribute is a typed element and, thus, it is associated to Type. Type constrains the sort of entities that can be assigned to slots representing that attribute in instances of their owning classifier. Since Classifier is a specialization of Type, we have that both Classes and Datatypes can be the associated types of an UML attribute. In other words, an attribute represents both an attribute function and a sort of a *relational image function*[69] that, for example, in binary relation *ownership* between the classifiers Person and Car, maps a particular Person p to all instances of Car that are associated with p via this relation (i.e., all cars owned by p). From a software design and implementation point of view, an attribute represents a method implemented by the owning class, and the type of the attribute

[69]In LINGO (Guizzardi & Falbo & Gonçalves, 2002), for instance, a relational image function is formally defined as follows: Let R be a binary relation defined for the two sets X and Y. The function **Im** with signature $\mathbf{Im(\_,\_): X \times (X \Leftrightarrow Y) \rightarrow \wp(Y)}$ is defined as $\mathbf{Im(x,R) = \{y \mid (x,y) \in R\}}$.

represents the returning type of that method. However, from a conceptual point of view, in the UML metamodel an attribute stands both for a monadic (instrinsic) and for a relational property and, thus, it can be considered a case of ***construct overload***. To eliminate this problem, we prescribe that attributes should only be used to represent attribute functions. As consequence, their associated types should be restricted to DataTypes only. Attributes whose associated types are classes (known as *mappings* in the object-orientation literature) (see for instance Bock & Odell, 1997b) are more important from a design and implementation point of view than from a conceptual one. We propose here a different treatment of relational properties, which is discussed latter in this section.

The DataType associated with an attribute A of class C is the representation of the quality structure that is the co-domain of the attribute function represented by A (postulate 6.2). In other words, a quality structure is the ontological interpretation of the UML DataType construct. Moreover, we have that a multidimensional quality structure (quality domain) is the ontological interpretation of the so-called *Structured DataTypes* (postulate 6.3).

Quality domains are composed of multiple integral dimensions. This means that the value of one dimension cannot be represented without representing the values of others. The fields of a datatype representing a quality domain QD represent each of its integral quality dimensions. Alternatively we can say that each field of a datatype should always be interpreted as representing one of the integral dimensions of the QD represented by the datatype. The constructor method of the datatype representing a quality domain must reinforce that its tuples always have values for all the integral dimensions. Finally, an algebra can be defined for a DataType so that the relations constraining and informing the geometry of represented quality dimensions are also suitably characterized.

UML offers an alternative notation for the representation of attributes, namely, *navigable end names*. That is, the same ontological concept (*attribute function*) is represented in the language via more than one construct, which characterizes ***construct redundancy***. This situation could be justified from a pragmatic point of view if navigable ends were used to model only structured DataTypes (as in figure 8.7), and if the textual notation for attributes were only used to model the simple ones (e.g., figure 8.6.a). However, as exemplified in figure 8.6, there is no constraint on using both notations for both purposes. To eliminate the potential ambiguity of this situation, we propose to use navigable ends to represent only attribute functions whose co-domains are *multidimensional quality structures* (quality domains). Conversely, those functions whose co-domains are quality dimensions should only be represented by the attributes textual notation.

According to the UML specification, *"a dataType is a type whose values have no identity (i.e., they are pure values)…so two occurrences of the same value cannot be differentiated."* What exactly is meant by *"have no identity"* here? Take for instance, the DataType Integer. It is clear that the member values of this DataType have identity in the usual sense of the term, i.e., identity statements such as $(1=1)$ and $\neg(1=2)$ are not only meaningful but also determinate. However, it is also true that two occurrences of the integer 1 refer to exactly the same entity. So, a better statement would be that different occurrences of the same DataType value do not have a separate identity. A universal U (represented by a UML Class) is multiply exemplified in each of its numerically distinct instances. A DataType is an abstract entity that collects other abstract entities (*"pure values"*) that can be multiply referred. A DataType is therefore not a multiply instantiated universal but an individual (set) with other individuals as members.

The ontological category of *relations* can be mapped in a representation mapping onto the UML concept of *associations*. Relations can be *material* or *formal*. The latter in turn can be subdivided in basic formal relations (internal relations) and comparative formal relations. Alternatively we can classify relations in basic relations and derived relations. The latter includes both comparative formal relations (derived from the relations among certain qualities) and material relations (derived from relator universals).

Since class diagrams only represent universals, the only basic formal relations among the ones we have considered that should have a representation in these models are the relations of *characterization*, *mediation* and *derivation*. These concepts have no representation in the UML metamodel, which characterizes another case of ***construct incompleteness***. Once more we propose an extension of the UML metamodel of figure 8.5 to include associations (i.e., extensions of the UML association metaclass) to represent these ontological distinctions. Moreover, in a profile implementing this extended metamodel, we include stereotyped associations (i.e., the base class is the UML association) to represent these newly introduced formal relations. The relations of characterization and mediation are represented by the stereotyped associations «characterization» and «mediation», respectively. The relation of derivation has a special notation discussed later in this section.

Object universals are characterized by intrinsic moment universals. As previously discussed, in the modelling profile proposed in this section, quality universals are not explicit represented in conceptual specifications. In contrast, mode universals are represented via a class decorated with the «mode» stereotype. The formal relation of characterization that takes place between a mode universal and the universal it characterizes is explicitly represented by an association stereotyped as «characterization».

Associations stereotyped as «characterization» must have in one of its association ends a class stereotyped as «mode» representing the characterizing mode universal. Likewise, every «mode» must be an association end for one «characterization» relation.

The *characterization* relation between a mode universal and the universal it characterizes is mapped at the instance level onto an *inherence* relation between the corresponding individual modes and their bearer objects. That means that every instance *m* of a class *M* stereotyped as «mode» is existentially dependent of an individual *c*, which is an instance of the class *C* related to *M* via the «characterization» relation. Inherence has the following characteristics: (a) it is a sort of existential dependence relation; (b) it is a binary formal relation; (c) it is a functional relation. These three characteristics impose the following metamodel constraints on the «characterization» construct:

1. by (a) and (c), the association end connected to the characterized universal must have the cardinality constraints of one and exactly one (lower = upper = 1);
2. by (a), the association end connected to the characterized universal must have the meta-attribute (isreadOnly = true);
3. «characterization» associations are always binary associations (#memberEnd = #ownedEnd = 2).

As discussed in chapter 6, there are no optional properties. For this reason, the «characterization» relation must also have a minimum cardinality of *one* on the association end connected to the mode universal. For example, if the universal *Patient* is characterized by the mode universal *Symptom*, then every instance *x* of patient must bear an instance *y* of symptom. As a consequence we have the following additional constraint:

4. In a «characterization» relation, the association end connected to the characterizing quality universal must have the minimum cardinality constraints of one (lower = 1).

An analogous case can be made for attributes. For example, since every apple bears a color quality, and every color quality has a value on the color domain, ergo, the corresponding *attribute function* is a total function. Thus,

5. A property owned by a classifier (representing an attribute of that class) must have the minimum cardinality constraints of one (lower = 1).

Finally, this constraint also holds for DataTypes. As previously mentioned, the fields of a structured DataType represent the integral

quality dimensions that compose the quality domain represented by this DataType. By its very definition, a quale in a quality domain must have values for all integral dimensions that compose that domain.

As discussed in section 6.3.3, the association class construct in UML exemplifies a case of ***construct overload***, since "an associaton class can have as instances either (a) a n-tuple of entities which classifiers are endpoints of the association; (b) a n+1-tuple containing the entities which classifiers are endpoints of the association plus an instance of the objectified association itself" (Breu et al., 1997). This is to say that an association class can be interpreted both as a *relational universal* and as a *factual universal*. In addition to that, since the *"instance of the objectified association itself"* is supposed to be an object identifier for the n-tuple, one cannot represent cases in which the same relator mediates multiple occurrences of the same n-tuple. Therefore, association classes on one hand represent a case of construct overload, on the other land, allow for a case of construct incompleteness at the instance level. We propose, therefore, to disallow the use of association classes in UML for the purpose of conceptual modeling.

In contrast, we propose to represent relational properties explicitly. We use the stereotype «relator» to represent the ontological category of relator universals. Relator universals can induce material relations. For instance, if there is a particular *Marriage m* connecting two persons *a* and *b*, we then can say that these persons stand up in a *married-to* relation, or that *derived-from([a,b],m)* holds. We say that a *derivation* relation holds between two universals F and G iff F is material relation (relational universal); G is a relator universal; For every instance *x* of F there is an instance y of G such that *derived-from(x,y)* holds.

A material relation induced by a relator universal R is represented by a UML association stereotyped as «material» (UML base class *association*). The basic formal relation *derivation* is represented by a dashed line with a black circle in one of the ends. A derivation relation is a specialized type of relationship between the stereotyped association representing the derived «material» association and the stereotyped class representing the founding «relator» universal. The black circle represents the role of foundation of the relator universal side. Every «material» association must be the association end of exactly one derivation relation.

The use of a different style of concrete syntax for the derivation relation than the one adopted for the other formal relations could in principle introduce some pragmatic inefficient, since the use of different syntax could allow for the misleading *implicature* (see chapter 2) that the represented entity belongs to a different ontological category. However, we believe that the drawback of this choice could be compensated by the benefit of using a syntax which is already familiar to UML users.

Derivation is a (total) functional relation and also a type of existential dependency. For this reason we have that every n-tuple *x* instance of a material relation F is derived from exactly one relator individual r and existentially dependent on the latter. As a consequence, the black circle end of the derivation relation must obey the following constraints: (a) have the cardinality constraints of one and exactly one (lower = upper = 1); (b) have the (isreadOnly = true).

The formal *mediation* relation between a relator universal and the universals it mediates is mapped onto the instance level to a *mediation* (*m*) relation between the corresponding individuals. That means that every instance r of a class R stereotyped as «relator» is existentially dependent of the individuals $c_1 \ldots c_n$ instances of the classes $C_1 \ldots C_2$ connected to R via the «mediation» relation. Mediation (as much as inherence) is a type of (binary) existential dependence relation, which imposes the following constraints on its representation:

1. the association ends owned by each of the mediated universals must have the minimum cardinality constraints of at least one (lower = 1);
2. the association ends owned by of the mediated universals must have the (isreadOnly = true);
3. «mediation» associations are always binary associations (#memberEnd = #ownedEnd = 2).

Moreover, since a relator is dependent (mediates) on at least two numerically distinct entities, we have the following additional constraint:

1. Let R be a relator universal and let $\{C_1 \ldots C_2\}$ be a set of universals mediated by R (related to R via a «mediation» relation). Finally, let $lower_{Ci}$ be the value of the minimum cardinality constraint of the association end connected to $C_i$ in the «mediation» relation. Then, we have that

$$(\sum_{i=1}^{n} lower_{Ci}) \geq 2 \ .$$

Let us suppose that R is a relator universal and that a mediation relation holds between R and the universals F and G (see figure 8.10). This means that for every instance r::R there is at least one f::F and one g::F such that *m(r,f)* and *m(r,g)*. Now, let H be the material relation between F and G derived from R. Then we have that for every r::R there is at least one pair [f,g]::H such that *derived-from([f,g],r)* holds. As a direct consequence, the association end connected to H in the *derivation* relation must have the minimum cardinality constraint of one (lower = 1).

Still on figure 8.10, from the cardinality constraints of the two «mediation» relations we can derive the maximum cardinality of the derivation relation (on the material relation end) and the cardinality constrains on both association ends of the material relation itself. For instance, the upper constraint $\delta$ on the end connected to G in the H relation is the result of $(d \times h)$; the upper constraint $\beta$ in the end connected to F is the result of $(f \times b)$. The upper constraint $\phi$ in the end H of the derivation relation is the result of $(b \times h)$. Likewise, we can calculate the derived minimum cardinality constraints in the following manner: $\gamma = c \times g$; $\alpha = e \times a$, and $\varepsilon = a \times g$.

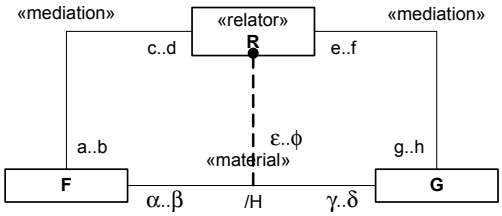



*Figure 8-10* Material Relations and their founding relators (the cardinality constraints of the derived relation and the derivation relation itself can be calculated from the corresponding mediation relations involving the founding relators)

Two alternative versions of a concrete example of this situation are depicted in figures 8.11.a and 8.11.b below. However, due to the lack of expressivity of the traditional UML association notation, these two models seem to convey the same information (from the perspective of the material relation *supervised-by*), although they describe completely different conceptualizations. As discussed in section 6.3.3, the benefits of explicitly representing relator universals instead of merely representing material relations, becomes even more evident in n-ary relations with $n > 2$.

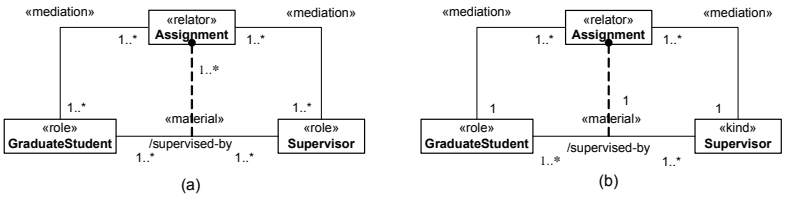



*Figure 8-11* Examplification of how relators can disambiguate two conceptualizations that in the standard UML notation would have the same interpretation

Once more we should highlight that the relator individual is the actual instantiation of the corresponding relational property (the objectified relation). Material relations stand merely for the facts derived from the relator individual and its mediating entities. Therefore, we claim that the representation of the relators of material relations must have primacy over the representation of the material relations themselves. In other words, the representation of «material» relations can be omitted but whenever a

«material» is represented it must be connected to an association end of a derivation relation.

We use the stereotype «formal» to represent domain formal relations. Comparative formal relations and material relations are derived relations. Whilst the former are derived from intrinsic properties of the related entities, the latter are derived from relational properties and their mediating entities. Therefore, we prescribe that «material» must have the meta-attribute (isDerived = true). Analogously, we use the meta-attribute (isDerived = true) to represent formal relations which are not *internal relations*, i.e., which are comparative.

Once more we should emphasize that there are no optional properties, e.g., there is nothing which has the optional property of *being married*. For this reason, «mediation» relations must have a minimum cardinality of one on the association end connected to the relator universal. For example, in figure 8.11, since the universal Supervisor is mediated by the relator universal Assignment, then every instance *x* of supervisor must have a *m(y,x)* relation with an instance *y* of assignment. A direct consequence of this rule is that the minimum cardinality constraints on both ends of the *supervised-by* material relation (derived from Assignment) must also have a value of at least one. The same applies to the association end connected to the material relation (*supervised-by*) in the derivation relation.

This is especially evident when the mediated entities are *Role* classifiers. Take, for instance, the examples of figure 8.11. As discussed in chapter 4, a role is a restriction of kind by means of a relational restriction condition. For instance, *a supervisor is a Person who supervises graduate students*. Thus, being related to an assignment is part of the very definition of the universal supervisor (in this conceptualization). As we have elaborated in chapter 7, for *x* to be a role individual, then there must a *role qua individual y* such that *i*(y,x). However, a role qua individual must be part of a relator. In fact, if *x* is an individual such that there is a qua individual *y* which inheres in *x* and which is part of a relator *z* then x is mediated by *z* (see chapter 6). As a consequence, we have that for every role individual *x* there must be a relator individual *z* that mediates *x*.

We can thus define the following constraints:

1. In a «mediation» relation, the association end connected to the relator universal must have the minimum cardinality constraints of one (lower = 1);
2. Every *relationally dependent* entity (role and role mixins) must be connected to an association end of a «mediation» relation.

Finally, in figure 8.5, we have a last subtype of classifier, namely, the meta-construct Interface. According to the UML specification, an interface

is a declaration of a coherent set of features and obligations. It can be seen as a kind of contract that partitions and characterizes groups of properties which must be fulfilled by any instance of a classifier that implements that interface. In an interpretation mapping from the UML metamodel to the ontology of figure 8.9, an interface qualifies as a case of ***construct excess***. This means that, being merely a design and implementation construct, there is no category in the conceptual modeling ontology proposed here that serve as the ontological interpretation for the UML interface. For this reason, we propose that the use of this construct should be disallowed in an ontologically well-founded version of UML that is suitable for conceptual modeling.

In this respect we therefore strongly disagree with the works of (Steimann, 2000a,b, 2001) that proposes that the ontological concept of roles can be the interpretation for the UML interface construct. As recognized by the UML specification itself, interfaces are merely declarations and, hence, *non-instantiable* model elements. Interfaces can be implemented by classes that obey different principles of identity. Thus, as a classifier, an interface can only be interpreted as a non-sortal. So, at most, it would be fair to propose the mapping from interface to the ontological category of *role mixin*. However, unlike role mixins, interfaces are neither necessarily relationally dependent nor anti-rigid. In summary, there is absolutely no ground to propose a (representation) mapping from an anti-rigid, relationally dependent sortal to a non-sortal which is not necessarily any of these things.

In figure 8.12, we present a revision of the UML metamodel of figure 8.5 that faithfully represents the ontological distinctions proposed in this section. All the different dependency relations discussed in this section, namely, *characterization*, *mediation* and *derivation* have in common that they are all directed binary relations. By distinguishing between domain associations and formal relations (represented by the *directed binary relationship* metaclass) in the extended metamodel we help to prevent the construction of *unintended models*, i.e., models that portrait state of affairs that are excluded by the underlying conceptualization (see chapter 3). In particular, we exclude the possibility of constructing models containing dependency relations with more than two association ends. In addition, all dependency relations on this meta-model stand for relations of *existential dependence* (i.e., *rigid specific dependence*). This is to say that, at the instance level, the dependent instance must be related to one and the same depended entity one in all possible worlds. As a consequence, the *target* meta-attribute of Dependency Relations in this meta-model is a *readOnly* meta-attribute, i.e., once assigned it cannot be modified. Finally, it is important to emphasize that this metamodel omits those elements which

have no ontological interpretation (*interface*), or which are ill-defined from an ontological point of view (*association class*).

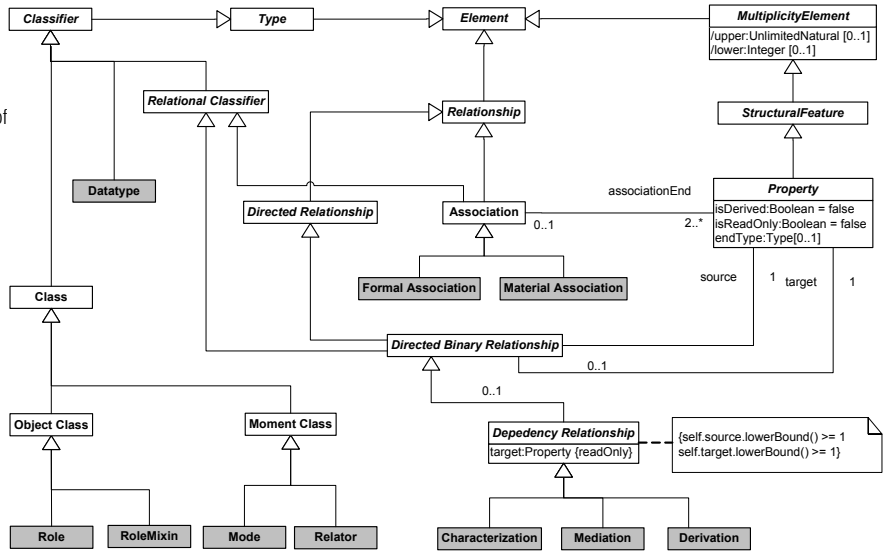



*Figure 8-12* Revised Fragment of the UML 2.0 metamodel according to the ontological categories of figure 8.9

The UML metamodel elements proposed in this revised model together with their characterizing specific syntactical constraints are implemented in a modelling profile. This profile, which extends the one of table 8.1, is summarized in tables 8.2 and 8.3 below.

*Table 8-2* Extension to the UML modelling profile of table 8.1 that implements the revised metamodel of figure 8.12

| Stereotype | Base Class | Description |
|---|---|---|
| «role» **A** | Class | Refinement of the definition of the «role» stereotype presented in table 8.1. |
| **Constraints** | | |
| 1. Refinement of condition 2 of «role» stereotype on table 8.1: Every «role» class must be connected to an association end of a «mediation» relation; | | |

| Stereotype | Base Class | Description |
|---|---|---|
| «roleMixin» ***A*** | Class | Refinement of the definition of the «roleMixin» stereotype presented in table 8.1. |
| **Constraints** | | |
| 1. Refinement of condition 1 «roleMixin» stereotype on table 8.1: Every «roleMixin» class must be connected to an association end of a «mediation» relation; | | |

| Stereotype | Base Class | Description |
|---|---|---|
| «mode» **A** | Class | A «mode» universal is an intrinsic moment universal. Every instance of mode universal is existentially dependent of exactly one entity. Examples include skills, thoughts, beliefs, intentions, symptoms, private goals. |

**Constraints**

1. Every «mode» must be (directly or indirectly) connected to an association end of at least one «characterization» relation;

| Stereotype | Base Class | Description |
|---|---|---|
| «relator» **A** | Class | A «relator» universal is a relational moment universal. Every instance of relator universal is existentially dependent of at least two distinct entities. Relators are the instantiation of relational properties such as marriages, kisses, handshakes, commitments, and purchases. |

**Constraints**

1. Every «relator» must be (directly or indirectly) connected to an association end on at least one « mediation » relation;

2. Let R be a relator universal and let $\{C_1 \ldots C_2\}$ be a set of universals mediated by R (related to R via a «mediation» relation). Finally, let $\text{lower}_{Ci}$ be the value of the minimum cardinality constraint of the association end connected to $C_i$ in the «mediation» relation. Then, we have that

$$(\sum_{i=1}^{n} \text{lower}_{Ci}) \geq 2 .$$

| Stereotype | Base Class | Description |
|---|---|---|
| «mediation» | Dependency Relationship | A «mediation» is a formal relation that takes place between a relator universal and the endurant universal(s) it mediates. For example, the universal *Marriage* mediates the role universals Husband and Wife, the universal *Enrollment* mediates Student and University, and the universal *Covalent Bond* mediates the universal Atom. |

**Constraints**

1. An association stereotyped as «mediation» must have in its source association end a class stereotyped as «relator» representing the corresponding relator universal (self.source.oclIsTypeOf(Relator)=true);

2. The association end connected to the mediated universal must have the minimum cardinality constraints of at least one (self.target.lower ≥ 1);

3. The association end connected to the mediated universal must have the property (self.target.isreadOnly = true);

4. The association end connected to the relator universal must have the minimum cardinality constraints of at least one (self.source.lower ≥ 1).

5. «mediation» associations are always binary associations.

| Stereotype | Base Class | Description |
|---|---|---|
| «characterization» | Dependency Relationship | A «characterization» is a formal relation that takes place between a mode universal and the endurant universal this mode universal characterizes. For example, the universals *Private Goal* and *Capability* characterize the universal *Agent.* |

**Constraints**

1. An association stereotyped as «characterization» must have in its source association end a class stereotyped as «mode» representing the characterizing mode universal (self.source.oclIsTypeOf(Mode)=true);

2. The association end connected to the characterized universal must have the cardinality constraints of one and exactly one (self.target.lower = 1 and self.target.upper = 1);

3. The association end connected to the characterizing quality universal (source association end) must have the minimum cardinality constraints of one (self.source.lower ≥ 1);

4. The association end connected to the characterized universal must have the property (self.target.isreadOnly = true);

5. « characterization » associations are always binary associations.

| Stereotype | Base Class | Description |
|---|---|---|
| Derivation Relation  | Dependency Relationship | A derivation relation represents the formal relation of derivation that takes place between a material relation and the relator universal this material relation is derived from. Examples include the material relation *married-to*, which is derived from the relator universal *Marriage*, the material relation *kissed-by*, derived from the relator universal *Kiss*, and the material relation *purchases-from,* derived from the relator universal *Purchase*. |

**Constraints**

1. A derivation relation must have one of its association ends connected to a relator universal (the black circle end) and the other one connected to a material relation (self.target.oclIsTypeOf(Relator)=true, self.source.oclIsTypeOf(Material Association)=true);

2. derivation associations are always binary associations;

3. The black circle end of the derivation relation must have the cardinality constraints of one and exactly one (self.target.lower = 1 and self.target.upper = 1);

4. The black circle end of the derivation relation must have the property (self.target.isreadOnly = true);

5. The cardinality constraints of the association end connected to the material relation in a derivation relation are a product of the cardinality constraints of the «mediation» relations of the relator universal that this material relation derives from. This is done in the manner previously shown in this section. However, since «mediation» relations require a minimum cardinality of one on both of its association ends, then the minimum cardinality on the material relation end of a derivation relation must also be ≥ 1 (self.source.lower ≥ 1).

| Stereotype | Base Class | Description |
|---|---|---|
| «material» | Association | A «material» association represents a material relation, i.e., a relational universal which is induced by a relator universal. Examples include *student studies in university*, *patient is treated in medical unit*, *person is married to person*. |

| Constraints |
|---|
| 1. Every «material» association must be connected to the association end of exactly one derivation relation;<br><br>2. The cardinality constraints of the association ends of a material relation are derived from the cardinality constraints of the «mediation» relations of the relator universal that this material relation is derived from. This is done in the manner shown in this section. However, since «mediation» relations require a minimum cardinality of one on both of its association ends, then the minimum cardinality constraint on each end of the derived material relation must also be ≥ 1 ;<br><br>3. Every «material» association must have the property (isDerived = true). |

| Stereotype | Base Class | Description |
|---|---|---|
| «formal» | Association | A «formal» association represents a formal relation, i.e., either a comparative relation (derived from intrinsic properties of the relating entities), or an internal relation. Examples include Person *is older than* Person, an Atom *is heavier than* another atom. We use UML symbolic representation for *derived* relations ( / ) to represent comparative relations. |

*Table 8-3* A constraint on properties representing attributes which is incorporated in this modelling profile

| Metaclass | Description |
|---|---|
| Property | An attribute in the UML metamodel is a property owned by a classifier. Attributes are used in this profile to represent attribute functions derived for quality universals. Examples are the attributes color, age, and startingDate. |
| **Constraints** | |
| 1. A property owned by a classifier (representing an attribute of that classifier) must have the minimum cardinality constraints of one (self.lower ≥ 1). | |

Finally, in the UML 2.0, it is possible to apply a profile to a model, and indicate whether the model only have stereotypes of the profile or whether elements of the reference metamodel (e.g., the metamodel of figure 8.5) are still allowed. This option can be used in models adopting the profile proposed in this section to disallow the use of interfaces and association classes.

## 8.3 Aggregation and Composition

### 8.3.1 The UML Metamodel

We refer once again to the metamodel fragment depicted in figure 8.5, but now with the focus on the representation of part-whole relations in UML.

A part-whole relation is specified in UML in the following manner: *"An association may represent an aggregation; that is, a whole/part relationship. In this case, the association-end attached to the whole element is designated, and the other association-end of the association represents the parts of the aggregation. Only binary associations may be aggregations."*

Every member end of an association (being a property) has an attribute named *aggregation*. This attribute indicates whether that association represents a part-whole relation and, if so, which kind of part-whole relation. The possible values that this attribute can assume are specified in the enumeration *AggregationKind*. They are:

1. **none:** indicates that the property has no aggregation, i.e., that the association that has this property as a member end does not represent a part-whole relation. This is the default value for this property;

2. **shared:** indicates that the property has a shared aggregation and, consequently, that the corresponding association represents a shareable part-whole relation;

3. **composite:** indicates that the property has a non-shared aggregation and, consequently, that the corresponding association represents a non-shareable part-whole relation (composition). Composition is represented by the *isComposite* meta-attribute on the part end of the association being set to true.

The UML specification defines the following semantics for composition: "*Composite aggregation is a strong form of aggregation, which requires that a part*

*instance be included in at most one composite at a time and that the composite object has sole responsibility for the disposition of its parts. This means that the composite object is responsible for the creation and destruction of the parts. In implementation terms, it is responsible for their memory allocation. If a composite object is destroyed, it must destroy all of its parts. It may remove a part and give it to another composite object, which then assumes responsibility for it. If the multiplicity from a part to composite is zero-to-one, the composite may remove the part, and the part may assume responsibility for itself, otherwise it may not live apart from a composite."*

Conversely, a shareable aggregation is defined as follows: "*A shareable aggregation denotes weak ownership; that is, the part may be included in several aggregates and its owner may also change over time. However, the semantics of a shareable aggregation does not imply deletion of the parts when an aggregate referencing it is deleted."*

In terms of primary meta-properties, *"both kinds of aggregations define a transitive, antisymmetry relationship; that is, the instances form a directed, non-cyclic graph. Composition instances form a strict tree (or rather a forest)."*

A part-whole relation is by default expressed as an aggregation in the UML. Otherwise, if the parts in the part-whole relation are non-shareable, then the relation is expressed as a composition (black-diamond). The use of composition implies the maximum cardinality of 1 w.r.t. the whole. An example of part-whole relation with shareable parts is the relation between researchers and research groups (researchers can be part of several research groups) depicted in figure 8.13.a. Conversely, the relation between a person and her heart (e.g., figure 8.13.b) is an instance of a non-shareable part-whole relation[70]. Finally, figure 8.13.c depicts a case of non-shareable parthood with optional wholes.

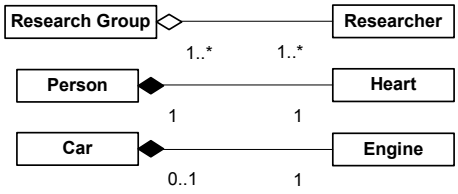



*Figure 8-13* Part-Whole relations with: (a) shareable part; (b) non-shareable part and mandatory whole; (c) mandatory part and optional whole

[70] We assume here a conceptualization in which a human heart cannot be shared by more than one human body, thus, excluding the case of Siamese Twins. Once more, the example is used here for illustration purposes only and not to defend a particular ontological commitment.

### 8.3.2 Ontological Interpretation and Re-Design

In spite of being a small fragment of the UML metamodel, this part relative to the modeling of part-whole relations poses a number of serious difficulties from a conceptual modelling point of view.

To start with, UML defines only one general sort of parthood relation, which can, in turn, be qualified in two types, namely aggregation and composition. This is because the UML metamodel considers only one equally general notion of objects, i.e., it is insensible to the ontological distinctions among types of substantial individuals.

As discussed extensively in chapter 5, in our ontology we consider four sorts of conceptual parthood relations, based on the type of entities they relate, namely, (a) **subQuantityOf**, which relates individuals that are *quantities*; (b) **subCollectionOf**, which relates individuals that are *collectives*; (c) **memberOf**, which relates individuals that are *functional complexes* or *collectives* as parts of individuals that are *collectives*; (d) **componentOf**, which relates individuals that are *functional complexes*.

These different sorts of conceptual parthood relations are depicted in Figure 8.14 below. These different relations between *objects* are specializations of a more abstract *partOf* relation defined to hold between *entities* in general.

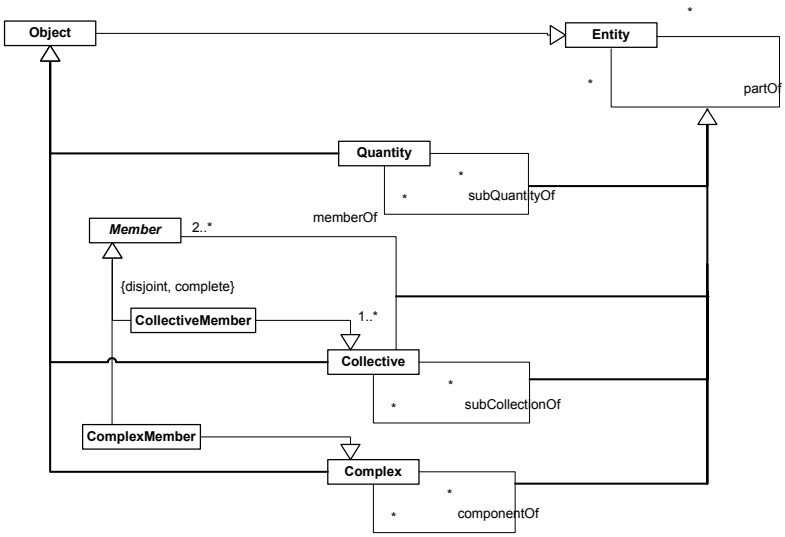



*Figure 8-14* Different types of meronymic relations according to the ontological category of their relata

In a first analysis, the general notion of aggregation in UML can be thought to represent the *partOf* relation in this ontology. In UML, the general aggregation is an *asymmetric* and *transitive* relation, but it is not said to be irreflexive. This is because, in contrast to the *partOf* relation, aggregation is a

relation between types, not between tokens. Therefore, it is, in principle, conceivable that we have a relation which is type-reflexive but which is still irreflexive at the instance level (i.e., there is no instance that bears this relation to itself).

The *partOf* relation is an *anti-symmetric* and *non-transitive* relation. Non-transitivity means that transitivity holds for certain cases but not for others. The relation of partOf must be deemed non-transitive since: two of its subclasses are transitive (subQuantityOf and subCollectionOf), one is intransitive (memberOf), and one is itself non-transitive (componentOf). In addition, this relation obeys the *irreflexivity* axiom and *weak supplementation principle* (i.e., if x is a part of y then there must be a z disjoint of x, which also part of y - see chapter 5).

We conclude then we cannot identify elements of the UML metamodel that can be used to represent neither the *partOf* relation nor any of its subclasses. Once more we have a case of ***construct incompleteness***. We therefore propose an extension of the UML 2.0 metamodel depicted in figure 8.5 to include metaclasses that represent each of these concrete types of conceptual parthood relations.

By far, the most commonly used of these relations in conceptual modeling is the *componentOf* relation. This is because *functional complexes* are also the most commonly represented entities in conceptual specifications. For this reason, we use the standard UML symbolic notation for aggregation to represent this relation.

As any *partOf* relation, *componentOf* is an irreflexive and anti-symmetric relation, which obeys the weak supplementation principle. As a result of this last principle, we have the following constraint: Let U be a universal whose instances are functional complexes and let $\{C_1 \ldots C_2\}$ be a set of universals related to U via aggregation relations. Finally, let $lower_{Ci}$ be the value of the minimum cardinality constraint of the association end connected to $C_i$ in the aggregation relation. Then, we have that

$$(\sum_{i=1}^{n} lower_{Ci}) \geq 2 \ .$$

*Mutatis Mutandis*, this constraint in fact holds for all types of *partOf* relations.

The *componentOf* is a non-transitive relation. Transitivity holds only in certain contexts, which can be isolated by following certain model patterns defined in sections 5.6 and 7.4.

In UML, only one of the secondary properties or part-whole relations is considered, namely *exclusiveness* (or *shareability*). A standard aggregation relation, represented as a hollow diamond in the association end representing the whole, stands for a non-exclusive (shareable) part-whole relation. The composition relation, represented as a black diamond in the

association end representing the whole, stands for an exclusive (non-shareable) one instead. Despite making this distinction, the UML specification is not precise on what exactly the distinction is supposed to represent. Firstly, the specification states that "*composite aggregation is a strong form of aggregation, which requires that a part instance be included in at most one composite at a time*". In the way it is stated, this sentence is simply wrong: since the composition relation is deemed transitive by the specification, an object x which is part of composite y is also part of any composite z of which y is part. Therefore, it cannot be a necessity for *"a part instance [to] be included in at most one composite at a time"*. Even if we ignore this problem as a formulation mistake, it is in principle still possible to interpret the non-shareability requirement as a demand for global exclusiveness. Global exclusiveness is defined as follows (definition 5.7): $(x <_x y) =_{df} (x < y) \wedge (\forall z\ (x < z) \rightarrow (z \leq y) \vee (y \leq z))$. This means that if x is an (globally) exclusive part of y, then if x is part of z (different from y) then either z is part of y, or y is part of z. As discussed in section 5.4.1, we adopt a weaker notion of exclusiveness, namely, one of exclusiveness w.r.t. to a given universal (or *local exclusiveness*). Therefore, if x of type X is an exclusive part of y of type Y, then there is no other z of type Y such that x is part of z. In summary, the composition relation in UML is taken here to represent an (locally) exclusive *componentOf* relation.

Another problem with the UML concept of composition is that it merges two different ontological meta-properties, namely, *exclusiveness* and *dependence*. This in itself is a case of ***construct overload***. However, the specification is even more ambiguous w.r.t. which kind of dependence relation is being considered. First, it states that "*the composite object is responsible for the creation and destruction of the parts… If a composite object is destroyed, it must destroy all of its parts.*" This seems to imply that there is a specific dependence from the parts to the composite, i.e., the parts are *existentially dependent* on the composite. If this were the case, the composite relation in UML would also represent a case of *inseparable parthood* (definition 5.15). Nonetheless, in another part of the text, it is specified that *"[the composite] may remove a part and give it to another composite object, which then assumes responsibility for it. If the multiplicity from a part to composite is zero-to-one, the composite may remove the part, and the part may assume responsibility for itself, otherwise it may not live apart from a composite."* This implies that if the minimum multiplicity from a part to composite is *one* then the part is only *generically dependent* on the universal that the composite instantiates, i.e., the part object can be part of any composite of the same type (part with mandatory whole). Conversely, if the minimum multiplicity from a part to composite is *zero-to-one* then the part is not at all dependent on the composite (part with optional whole).

Despite the possible dependency relations that might hold from the part to the whole, there are also (specific or generic) dependency relations that might hold from the whole to the parts. That is, a whole object can be existentially dependent on a specific part (essential), or generically dependent on the universal that a part instantiates (mandatory part).

Shareability and inseparability/essentiality are completely orthogonal meta-properties, i.e., there are shared parts which are essential (and/or inseparable), and there are exclusive (non-shareable) parts which are only mandatory. An example of the former is depicted in figure 5.33. In an instance of that model, a particular Lecture can be shared by a Regular Course and by a Studium Generale Course. Nonetheless, the lecture is an inseparable part of both courses. An example of the latter is depicted in figure 8.13.b. A heart is not an essential part of a particular human being. However, it is a non-shareable part.

Finally, as discussed in depth in section 7.2, only rigid universals can participate as wholes in essential parthood relations. In the case of anti-rigid universals, the modal necessity of the parts can only be a case of *de dicto* necessity. These parts are named *immutable parts* instead.

In order to eliminate the construct overload aforementioned we propose that shared/composition notation in UML to be used to represent only shareability. To represent mandatory parts and mandatory wholes we use the minimum cardinalities of the represented parthood relation in the corresponding way: a minimum multiplicity of one in the side of the part represents a generic dependence from the whole to the part (mandatory part); a minimum multiplicity of one in the side of the whole represents a generic dependence from the part to the whole (mandatory whole). Additionally, we extend the original UML aggregation notation with the following *tagged values:* (a) {*inseparable* = *true*}: represents inseparable parts (i.e., every part is existentially dependent of a specific whole); (b) {*essential* = *true*}: represents essential parts (i.e., every whole is existentially dependent of a specific part); (c) {*immutable* = *true*}: represents immutable parts (i.e., every whole bears a parthood relation to a specific part in all circumstances that it instantiates that specific whole universal). Since inseparability of parts is a stronger constraint than generic dependence, then whenever {*inseparable* = *true*} the minimum cardinality constraint in the association end connected to the whole must be one. Likewise, whenever {*essential* = *true*} or {*immutable* = *true*} the minimum cardinality constraint in the association end connected to the part must be one.

Finally, in definition 5.14, we have introduced the notion of an extensional individual, i.e., an individual for which all parts are essential. We represent a universal whose instances are extensional entities by a tagged value {*extensional*} on top of the classifier representing that universal. A natural constraint is that all parthood relations in which an {*extensional*}

classifier participates as a whole, must have the meta-property {*essential* = *true*}.

Although defined above for the *componentOf* relation, these meta-properties can also be used to qualify the other types of *partOf* relations. In the sequel, we analyze each of these relations separately.

**subQuantityOf:** As discussed in section 5.5.1, a *quantity* stands for a maximally-connected-amount-of-matter. Since a quantity is maximal, it cannot have as a part a quantity of the same kind. For the same reason, a *subQuantityOf* relation is always non-sharable. For example, take a case in which this relation holds between a quantity of alcohol *x* and a quantity of wine *y*. Since y is self-connected it occupies a self-connected portion of space. The same holds for x. In addition, the topoid[71] occupied by x must be a (improper) part of the topoid occupied by y. Now suppose that there is a portion of wine z (different from y) such that x is a subQuantityOf z. A consequence of this is that z and x overlap, and since they are both self-connected, we can define a portion of wine w which is itself self-connected. In this case, both z and x are part of w and therefore, they are not *maximally-self-connected-portions*. This contradicts the premises that x and z were quantities. Hence, we can conclude that the *subQuantityOf* (Q) relation is always non-sharable.

Since every part of a quantity is itself a quantity, Q-parthood must have a cardinality constraint of *one and exactly one* in the subquantity side. Take once more the alcohol-wine example above. Since alcohol is a quantity (and, hence, maximal), there is exactly one quantity of alcohol which is part of a specific quantity of wine.

As discussed in section 5.6, quantities are mereologically invariant, i.e., the change of any of its parts changes the identity of the whole. In other words, all parts of a quantity are essential. The consequences of this characteristic are: (i) *subQuantityOf* relations are essential parthood relations; (ii) since essential parthood relations are always transitive, *subQuantityOf* is always transitive (i.e., for all a,b,c, if Q(a,b) and Q(b,c) then Q(a,c)); (iii) since quantities are extensional entities, the *weak supplementation axiom* (3) defined to hold for all partOf relations can be replaced by the adoption of the *strong supplementation axiom* (chapter 5) for the case of the *subQuantityOf* relation.

The axiomatization of the *subQuantityOf* relation thus includes the basic axioms of any mereological theory, namely, irreflexivity, anti-symmetry and transitivity of the proper-part relation. But also the *strong supplementation axiom* and the *extensionality principle* (see section 5.1.2). Moreover, it includes

[71] A Topoid is a region of space with a certain mereotopological structure (Guizzardi & Herre & Wagner, 2002a).

the exclusive parthood (definition 5.9) and the essential parthood (definition 5.11) axioms.

In other words, the axiomatization of this relation is the one of *Extensional Mereology (EM)*, with non-shareable parts. That means that two quantities are the same iff they have the same parts, and no two instance of the same quantity kind overlap.

**subCollectionOf:** Like quantities, collectives are maximal entities. However, in contrast with quantities, the unifying relation of a collective is not necessarily one of physical connection. For this reason, a collective can be shared by two or more collectives. For instance, the crowd $C_1$ occupying street $St_1$ and the crowd $C_2$ occupying street $St_2$ can both have as a part (*subCollectionOf*) the crowd $C_3$ occupying the intersection between streets $St_1$ and $St_2$. Therefore, *subCollectionOf* (C) can be shareable in some case while non-shareable in others.

In this example, the crowds $C_3$ and $C_1$ (or $C_2$) are not unified by the same type of relation. In fact, the unifying relation of the former is a refinement of that of the latter. Since a collective is a maximal entity, it is not the case that a collective can have as part another collective of the same type (i.e., unified by the same relation). Moreover, for the same reason, any collective can have at maximum one subcollection of a given type. Finally, as discussed in section 5.5.2, since every subcollection of a collective is obtained by refining the unifying relation of the latter, the subCollectionOf relation is always transitive.

As discussed in section 5.6, unlike quantities, collectives do not necessarily have an extensional criterion of identity. That is, whereas for some collectives the addition or subtraction of a subcollection (or a member) renders a different individual, it is not the case that this holds for all of them.

In summary, if cardinality constraints are fully specified, then the C-parthood is as such that: (i) the cardinality constraints in the association end relative to the part is one and exactly one; (ii) only holds between collectives; (iii) it is transitive, i.e., for all a,b,c, if C(a,b) and C(b,c) then C(a,c). Every *subCollectionOf* relation is irreflexive, anti-symmetric, transitive and obeys the weak supplementation axiom. That is to say that the axiomatization of the *subCollectionOf* relation is the one of the *Minimum Mereology (MM)*.

**memberOf:** The *member-collection* relation is one that holds between a *singular entity* (either a complex or a collective considered as a unity) and a collective. As discussed in section 5.6, *memberOf* (M) relations are never transitive, i.e., they are *intransitive*. This is to say that for all a,b,c, if M(a,b) and M(b,c) then $\neg$M(a,c). Nonetheless, transitivity does hold across M and

C, i.e., if we have M(x,y) and C(y,z) then it is also the case that M(x,z). In other words, if an individual x is a *memberOf* a collective Y, which is itself a *subCollectionOf* collective Z, then x is also a *memberOf* Z.

Typically members can be shared by more than one collective. However, it is also conceivable to have non-shareable memberOf relations.

As previously mentioned, collectives are not necessarily extensional individuals. However, when this is the case, all *memberOf* relations that the extensional collective participates as a whole are relations of essential parthood. Since a collective (by definition) has a uniform structure, then all members of a collective are supposed to be indistinguishable. As a consequence, it cannot be the case that some members of a collection are essential and others not. In summary, *memberOf* relations are only relations of essential parthood if the collective in the association end connected to the whole entity is an extensional individual.

In figure 8.15 below we present a revision of the UML meta-model in order to faithfully represent the ontological distinctions discussed in this section. As previously discussed, the original UML metamodel only allows for the distinction among parthood relations w.r.t. the shareability meta-property (via the meta-attribute *aggregationKind*). In other words, parthood relations are treated as ordinary associations which can have the meta-property of being an aggregation or a composition. This is a poor modeling choice for several reasons. Firstly, because it does not make justice to the special meta-properties and semantics of parthood relations. Secondly, because it is imprecise on the meaning of non-shareability represented, which collapses both the representations for shareability and inseparability in one single construct. Finally, because it does not sufficiently constrain the metamodel, the current representation of part-whole relations in UML allows for the representation of state of affairs which are not truthful to the underlying conceptualization. For example, one can define a parthood association in which both association ends have incompatible values for the aggregationKind meta-attribute. Moreover, like dependency relations, parthood relations are directed binary relations and therefore cannot be allowed to have more than two association ends. However, in the original UML metamodel, one can define a parthood association with arity higher than two.

In our proposed revised metamodel, we separate the representation of standard domain association from meronymic associations, allowing for a faithful representation and special semantic treatment for the latter. Moreover, we explicitly represent the different kinds of meronymic relations (distinguished by the category of entities they relate) and provide mechanisms to fully qualify them in terms of the ontological meta-properties considered. Finally, we proscribe the use of standard UML associations to represent part-whole relations. In other words, we require

that in any implementation of this metamodel the meta-attribute *aggregation* of the UML metaclass Property must have the value *none*.

The metamodeling choices adopted (e.g., by making a *meronymic relationship* a special type of *directed binary relationship*) entails a cleaner metamodel and one that approximates the class of possible models to that of intended ones (see chapter 3).

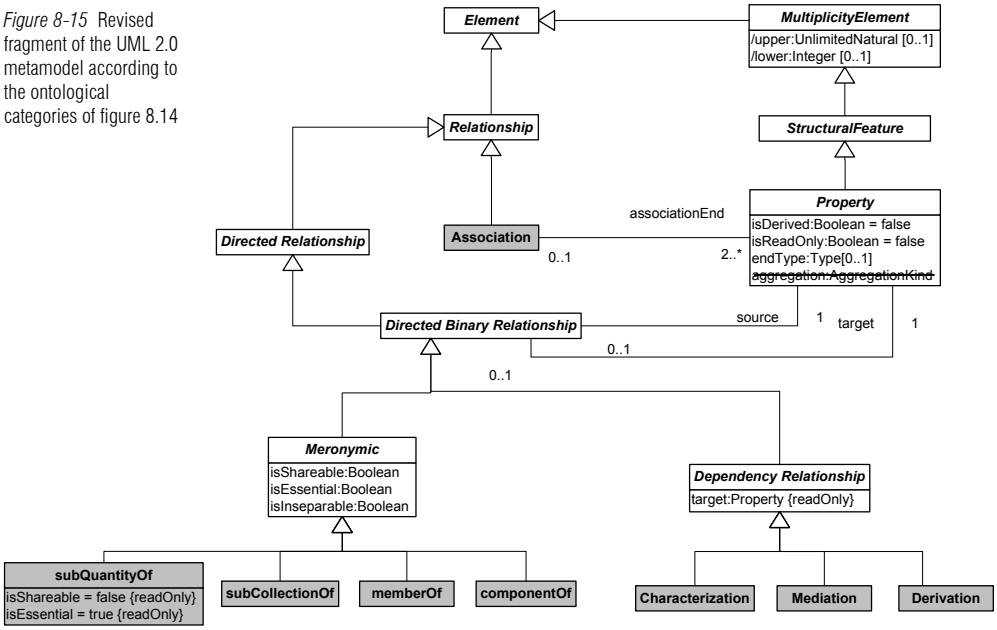



*Figure 8-15* Revised fragment of the UML 2.0 metamodel according to the ontological categories of figure 8.14

Once more, we have defined a modelling profile implementing the corresponding revised fragment of the UML metamodel. In table 8.4, we summarize the results of this section by presenting this profile that implements the leaf ontological distinctions among *meronymic* relations depicted figure 8.14, together with the syntatical constraints that characterize these relations. This profile extends the ones presented in tables 8.1 to 8.3.

*Table 8-4* Extensions to the UML profile of tables 8.1 to 8.3 which implement the revised metamodel of figure 8.15

| Metaclass | Description |
|---|---|
| Meronymic | Abstract metaclass representing the general properties of all meronymic relations. Meronymic has no concrete syntax. Thus, symbolic representations are defined by each of its concrete subclasses. |

Non-reflexivity, Anti-Symmetry, Non-Transitivity and Weak Supplementation.

**Constraints**

1. **Weak Supplementation:** Let U be a universal whose instances are wholes and let $\{C_1 \ldots C_2\}$ be a set of universals related to U via aggregation relations. Let $lower_{Ci}$ be the value of the minimum cardinality constraint of the association end connected to $C_i$ in the aggregation relation. Then, we have that

$$(\sum_{i=1}^{n} lower_{Ci}) \geq 2;$$

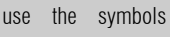
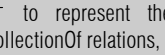
2. **Essential Parthood:** The *isEssential* attribute represents whether the meronymic relation is one of essential parthood, i.e., whether the part is essential to the whole. In case the classifier connected to the association end representing the whole is an anti-rigid classifier, then the meta-attribute *isEssential* must be false, whereas the meta-attribute *isImmutable* may be true. However, if *isEssential* is true (in case of a rigid classifier with essential parts) then *isImmutable* must also be true. The concrete representation of this meta-property is via the *tagged value* {essential} decorating the association;

3. **Inseparable Parthood:** The *isInseparable* attribute represents whether the meronymic relation is one of inseparable parthood, i.e., whether the whole is essential to the part. The concrete representation of this meta-property is via the tagged value {inseparable} decorating the association;

4. **Shareable Parthood:** The *isShareable* attribute represents whether the meronymic relation is (locally) shareable, i.e., whether the part can be related to more than a whole of that kind. The concrete representation of this meta-property is via the color property of the symbol used to depict this relation (a diamond with or without a decorating letter): if (isShareable = true) then the symbol is shown in white color, otherwise, it is shown in black.

| Metaclass | Description and Concrete Syntax |
|---|---|
| componentOf | componentOf is a parthood relation between two complexes. Examples include: (a) my hand is part of my arm; (b) a car engine is part of a car; (c) an Arithmetic and Logic Unit (ALU) is part of a Central Process Unit (CPU); (d) a heart is part of a circulatory system. Since this is by far the most used in conceptual modeling, we propose the use of the standard UML symbolic representation for aggregation/composition to represent this relation, i.e., we use the symbols  and  to represent the shareable and non-shareable componentOf relations, respectively. |
| **Meta-Properties** | |
| Non-reflexivity, Anti-Symmetry, Non-Transitivity and Weak Supplementation. | |
| **Constraints** | |
| 1. The classes connected to both association ends of this relation must represent universals whose instances are *functional complexes*. A universal X is a universal whose instances are functional complexes if it satisfies the following conditions: (i) If X is a sortal universal, then it must be either stereotyped as «kind» or be a *subtype* of a class stereotyped as «kind»; (ii) Otherwise, if X is a mixin universal, then for all classes Y such that Y is a *subtype* of X, we have that Y cannot be either stereotyped as «quantity» or «collective», and Y cannot be a *subtype* of class stereotyped as either «quantity» or «collective». | |
| **Metaclass** | **Description and Concrete Syntax** |
| subQuantityOf | subQuantityOf is a parthood relation between two quantities. Examples include: (a) alcohol is part of Wine; (b) Plasma is part of Blood; (c) Sugar is part of Ice Cream; (d) Milk is part of Cappucino. We propose the icon  to represent this relation. |
| **Meta-Properties** | |
| Non-reflexivity, Anti-Symmetry, Transitivity and Strong Supplementation (Extensional Mereology). | |

| Constraints |
|---|
| 1. This relation is always non-shareable (isShareable = false);<br><br>2. All entities stereotyped as «quantity» are extensional individuals and, thus, all parthood relations involving quantities are essential parthood relations;<br><br>3. The maximum cardinality constraint in the association end connected to the part must be one (self.target.upper = 1).<br><br>4. The classes connected to both association ends of this relation must represent universals whose instances are *quantities*. A universal X is a universal whose instances are quantities if it satisfies the following conditions: (i) If X is a sortal universal, then it must be either stereotyped as «quantity» or be a *subtype* of a class stereotyped as «quantity»; (ii) Otherwise, if X is a mixin universal, then for all classes Y such that Y is a *subtype* of X, we have that Y cannot be either stereotyped as «kind» or «collective», and Y cannot be a *subtype* of class stereotyped as either «kind» or «collective». |

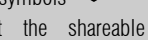
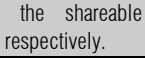
| Metaclass | Description and Concrete Syntax |
|---|---|
| subCollectionOf | subCollectionOf is a parthood relation between two collectives. Examples include: (a) the north part of the Black Forest is part of the Black Forest; (b) The collection of Jokers in a deck of cards is part of that deck of cards; (c) the collection of forks in cutlery set is part of that cutlery set; (d) the collection of male individuals in a crowd is part of that crowd. We use the symbols  and  to represent the shareable and non-shareable subCollectionOf relations, respectively. |

| Meta-Properties |
|---|
| Non-reflexivity, Anti-Symmetry, Transitivity and Weak Supplementation (Minimum Mereology). |

**Constraints**

1. The classes connected to both association ends of this relation must represent universals whose instances are *collectives*. A universal X is a universal whose instances are collectives if it satisfies the following conditions: (i) If X is a sortal universal, then it must be either stereotyped as «collective» or be a *subtype* of a class stereotyped as «collective»; (ii) Otherwise, if X is a mixin universal, then for all classes Y such that Y is a *subtype* of X, we have that Y cannot be either stereotyped as «kind» or «quantity», and Y cannot be a *subtype* of class stereotyped as either «kind» or «quantity».

2. The maximum cardinality constraint in the association end connected to the part must be one (self.target.upper = 1).

| Metaclass | Description and Concrete Syntax |
|---|---|
| memberOf | memberOf is a parthood relation between a complex or a collective (as a part) and a collective (as a whole). Examples include: (a) a tree is part of forest; (b) a card is part of a deck of cards; (c) a fork is part of cutlery set; (d) a club member is part of a club. We use the symbols  and  to represent the shareable and non-shareable memberOf relations, respectively. |

**Meta-Properties**

Non-reflexivity, Anti-Symmetry, Intransitivity and Weak Supplementation.
Although transitivity does not hold across two memberOf relations, an memberOf relation followed by subCollectionOf is transitive. That is, for all a,b,c, if memberOf(a,b) and memberOf(b,c) then ¬memberOf(a,c), but if memberOf(a,b) and subCollectionOf(b,c) then memberOf(a,c).

**Constraints**

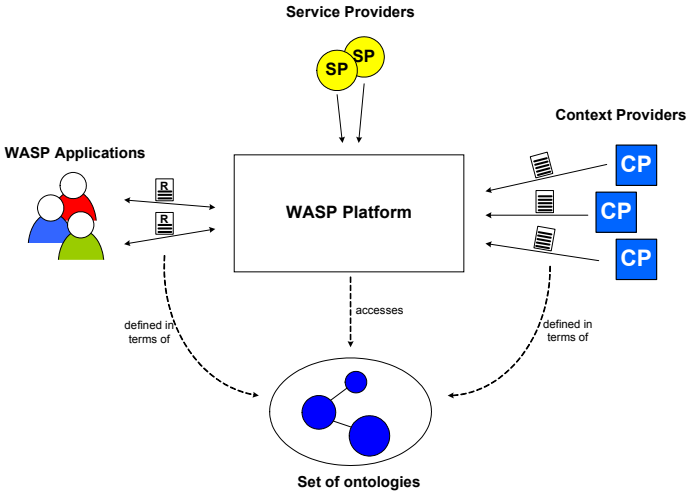
1. This relation can only represent essential parthood (*isEssential* = *true*) if the object representing the whole on this relation is an extensional (*isExtensional* = *true*) individual. In this case, all parthood relations in which this individual participates as a whole are essential parthood relations;

2. The classifier connected to association end relative to the whole individual must be a universal whose instances are *collectives*. The classifier connected to the association end relative to the part can be either a universal whose instances are collectives, or a universal whose instances are functional complexes.

## 8.4 An Application of the re-designed UML

In (Ríos, 2003), an architecture for an ontology-based context-aware service platform is proposed[72]. This platform, depicted in figure 8.16 below, employs distributed ontologies to define the semantics of syntactic items which are used to compose the messages exchanged by the platform and its environment. These messages include both context-aware applications service subscriptions and context-information supplied by external (context) providers.

*Figure 8-16* An ontology-based version of the WASP platform




As demonstrated by Ríos, the use of ontologies in this version of the WASP platform brings a number of important benefits to the original proposal (Costa, 2003). These benefits include:

1. *More intelligent behaviour* and the ability to reason about context information;
2. *Reusability*: the platform can (re)use already existing ontologies for the modeling of context information;
3. *Flexibility*: in contrast to the original proposal the platform is not closed w.r.t. a pre-defined set of context modeling concepts.

Due to these benefits, *ontologies* are being considered in practically all the architectural evolutions of this platform (Costa, 2004b; Santos, 2004).

[72] This platform is discussed in more detail in chapter 3 of this thesis. For a complete description of the proposed architecture one should refer to (Ríos, 2003).

In spite of these benefits, in (Ríos, 2003), the author discusses the insufficiency of *semantic web languages* (and the *lightweight ontologies* produced) to prevent interoperability problems when different ontologies are integrated. Ríos, proposes the following illustrative example on the integration of five independent domain ontologies.

The first ontology (which has a fragment depicted in figure 8.17) is a *Spatial ontology* that defines the concepts of *Spatial Location* and *Physical Object* and their corresponding properties (e.g., Spatial Location includes attributes such as *latitude* and *longitude* coordinates). This ontology might be considered as a very simple upper level ontology, as it does not define knowledge related to any specific domain. Thus, it can be referred or imported by different domain ontologies. For example, this ontology could be used by a GPS sensor agent to provide a service to track the location of physical objects in a context-aware platform.

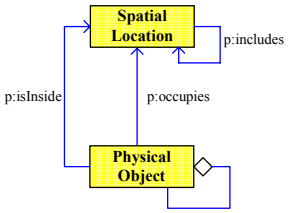



*Figure 8-17* Fragment of a Spatial Ontology (from Ríos, 2003)

There are two axioms defined for the *Spatial* ontology:

1. For every two arbitrary physical objects X and Y, if there are two spatial locations A, B, such that X occupies A, Y occupies B, and A is equal to B, then X and Y are the same physical object.

This axiom helps the users of this ontology to identify an object in a given time instant (*synchronic identity*). However, it cannot distinguish if two physical objects X and Y at different spatial locations in different time instants are the same objects (*diachronic identity*). For this reason, the ontology prescribes the following axiom.

2. For every two arbitrary physical objects X and Y, X is equal to Y if and only if they have the same parts, i.e., the *identity criterion* for physical objects is determined by the sum of its parts (*extensional identity criterion*).

A second ontology presented is a fragment of a *Medical* ontology (figure 8.18) defining some medically related concepts such as *Human Organ* or *Human Being* and *Surgery Room*. Ríos presents a situation, in which the

*Medical* ontology imports the concepts of *Spatial Location* and *Physical Object* from the *Spatial* ontology (symbolized by the i: character in the name of the class representing these concepts). The idea is to allow for the possibility of defining applications for checking location of patients, locate organs for transplants, and so forth.

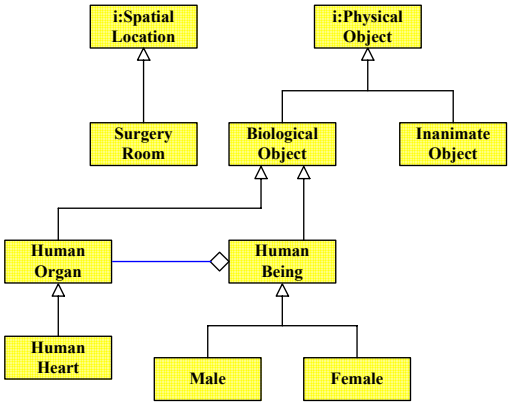



*Figure 8-18* Fragment of a Medical Ontology (from Ríos, ibid.)

A third ontology presented is shown in figure 8.19. The idea is to represent a fragment of *Legal* ontology, which represents legal aspects of people and that can be used by bureaucratic applications. This ontology imports the concepts of *Human Being*, *Male* and *Female* from the *Medical* ontology. This import allows, for example, legal applications to refer to the medical histories of people; to have access to their personal data (e.g., blood type, skin colour, fingerprints, height, weight); to differentiate people by sex; or to maintain a record of living and deceased people in a community.

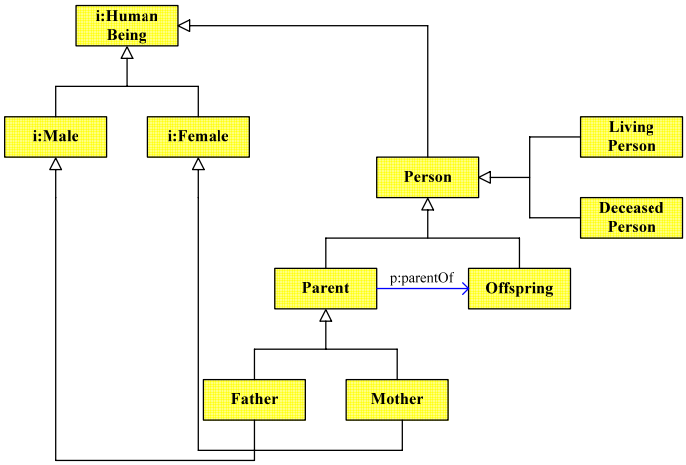



*Figure 8-19* Fragment of a Legal Ontology (from Ríos, ibid.)

Figure 8.20 shows a fragment of *Museum* ontology, which imports the *Spatial* and the *Medical* ontology to define spatial locations like galleries within a museum, or inanimate objects like statues. These imported ontologies allow for applications to locate objects within the museum (e.g., statues, paintings) using the *Museum* ontology.

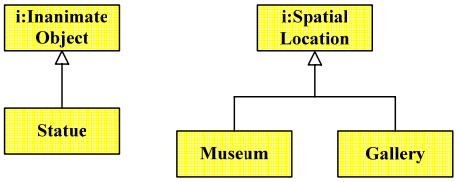



*Figure 8-20* Fragment of a Museum Ontology (from Ríos, ibid.)

Finally figure 8.21 represents a fragment of a Musical Ontology containing some related concepts. Ríos defines as an application for the complete ontology (to which this fragment belongs) an *Event Advisor*, which notifies users about upcoming events that match their personal interests. The Music ontology imports from the Legal ontology concepts like person (and its possible attributes, like name, age, sex, etc.).

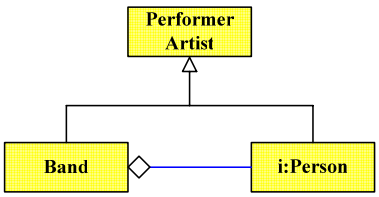



*Figure 8-21* Fragment of a Music Ontology (from Ríos, ibid.)

Ríos describes the following problems that can occur with the integration of these ontologies:

1. "An application using the *Medical* ontology can derive the following wrong information: if a human being receives a heart transplant, he/she becomes a different human being. This is due to the extensional identity criteria, which is defined for physical objects in the *Spatial* ontology. If the identity of an object is defined by the sum of its parts, then changing one of the parts changes the identity of the object. Similarly, consider a tourist route planner application that plans a route including tourist points of interest or events never seen by the user of the application. Due to an accident, a human statue known by the user has lost a hand. The application will consider this statue different from the one the user visited; therefore it will be included in the route plan by error. This example uses a physical object (statue) for the purpose of illustration of the problem, but an analogous situation can be imagined with events such as a play or a concert";

2. "Suppose an application for the obituary section of a music newspaper, which sends information about artists who die. It uses the Musical ontology, which imports the Legal ontology (to reuse the concept of person). The application will malfunction and it will send information about every person who dies, since [according to the ontology of figure 8.21] every person is a performer artist. The intention in the ontology represented in figure [8.21] is to represent that either persons or bands are performer artists. However, as a side effect, the ontology also states that every person is a performer artist";

3. "Since Musical ontology imports the Legal ontology, which imports the Medical ontology, the heart (and all other parts) of a person can be inferred to be part of a band, due to transitivity of the "*partOf*" relation, which can cause undesirable inferences to be derived".

A fragment of a *Music Ontology* such as the one presented in figure 8.21 can be found in practice in the *MusicBrainz II Metadata proposal*[73]. A simplified version of the MusicBrainz database structure is presented in figure 8.22.

[73] MusicBrainz is a large database of music metadata (http://www.musicbrainz.org/).

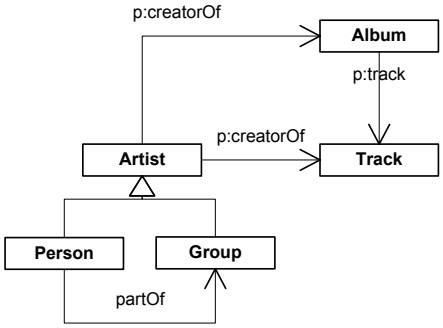



*Figure 8-22* Fragment of the *MusicBrainz* metadata proposal

In fact, this model excerpt can be seen as an extension of a more general pattern found in the *FOAF (Friend-of-a-Friend)* ontology[74] (Brickley & Miller, 2003) shown in figure 8.23 below. The FOAF ontology is a proposal for capturing concepts related to the representation of personal information and social relationships. Its purpose is to serve as a basis for developing computational support for online communities. The FOAF ontology is also used by the SOUPA ontology (see discussion below) to support the expression and reasoning about a person's profile and social connections in pervasive computing applications.

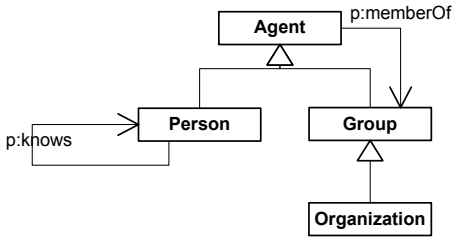



*Figure 8-23* Fragment of the *FOAF (Friend-of-a-Friend)* ontology

The conceptualization modelled in the fragment of figure 8.23 has an analogous representation in the *SOUPA (Standard Ontology for the Ubiquitous and Pervasive Application)*[75] Ontology (Chen, 2004; Chen et al., 2004). The SOUPA ontology also includes a Spatial Ontology (such as the one of figure 8.17), whose fragment is presented in figure 8.24.

[74] See The FOAF Project (http://www.foaf-project.org/) and the FOAF Vocabulary Specification (http://xmlns.com/foaf/0.1/)
[75] http://pervasive.semanticweb.org.

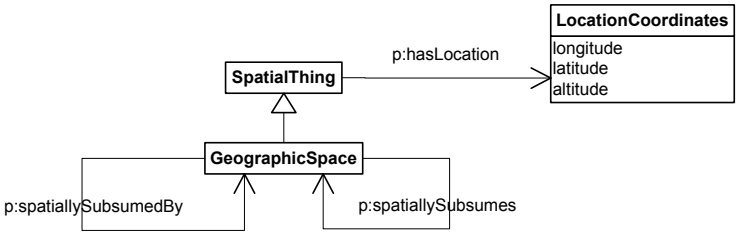



*Figure 8-24* Fragment of the *SOUPA (Standard Ontology for the Ubiquitous and Pervasive Applications)* dealing with spatial concepts and relations

SOUPA is a proposal for a standard ontology for supporting pervasive and ubiquitous computing application. It integrates parts of several other ontologies such as FOAF, DAML-Time (Hobbs, 2002; Pan & Hobbs, 2004), OpenCyC[76] and OpenGIS (Cox et al., 2003) (Spatial Entities), Rei Policy ontology (Kagal & Finin & Joshi, 2003), and COBRA-ONT (Chen & Finin & Joshi, 2004), but also an ontology defining agent related concepts named *MoGATU BDI* ontology (see figure 8.25)[77].

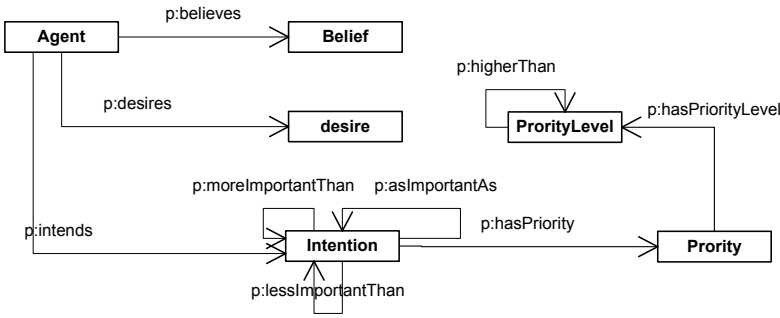



*Figure 8-25* Fragment of the *MoGATU BDI* ontology

In the sequel, we apply the ontologically well-founded version of UML redesigned in this chapter, together with the modelling techniques proposed throughout this thesis, to illustrate the use of the language in making explicit the underlying ontological commitments and in producing an adequate conceptual model representation that integrates the ontologies used in the examples of (Ríos, 2003) as well as the fragments of MusicBrainz II, FOAF, SOUPA and MoGATU BDI presented above.

The result of redesigning the integrated model is shown in figure 8.26 below.

[76] http://www.opencyc.org/.
[77] http://mogatu.umbc.edu/bdi/.

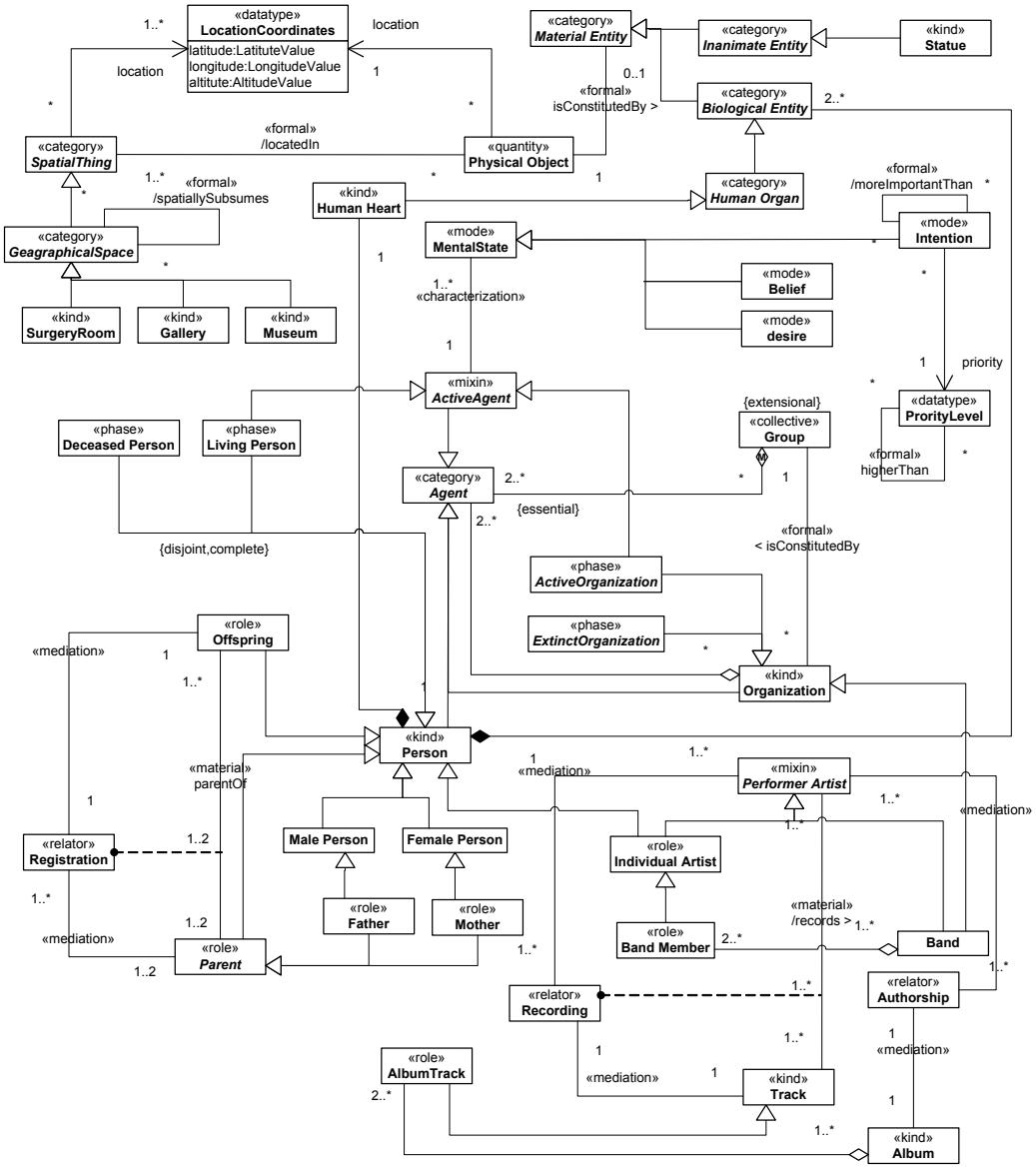



*Figure 8-26* An integrated ontology using the ontologically well-founded version of UML proposed in this thesis

In producing this conceptual specification we have been forced to make a number of assumptions. This is due to the lack of information provided by the integrated ontologies w.r.t. the real-world semantics of the concepts represented. We emphasize, nonetheless, that the goal here is to demonstrate the suitability of the modelling language proposed. Thus, the underlying conceptualization which results from the set of assumptions made is of lesser relevance.

First of all, in the model of figure 8.26, we separate physical spaces such as Surgery Room, Gallery and Museum from their spatial location (as informed by a GPS system), as it is the case in the SOUPA ontology. Firstly, because some geographical spaces (e.g., university) can have several location coordinates, but also because different geographical spaces can be associated with a particular set of location coordinates in different circumstances. Thus, whereas the physical spaces are represented by the general category of spatial thing, the spatial location of these physical spaces is modelled here as a quality domain composed of the quality dimensions *latitude*, *longitude* and *altitude*. The relation *includes* between spatial locations in figure 8.17 is represented by the relation *spatiallySubsumes* between *geographical spaces* in figure 8.26.

A *physical object* is said to be *located in* (*isInside*) a geographical space if the location of the former is included in that of the latter. Additionally, we assume that, in contrast with physical objects, *Biological Entities*, *Persons* and *Inanimate Entities* do not carry extensional principles of identity. Therefore, we differentiate a *Biological Entity*, such as a heart, from the quantity of cellular tissue that constitutes this entity. Likewise, we differentiate an *inanimate object*, such as a statute, from the raw material that constitutes it (e.g., a lump of clay).

A person is composed of a number of biological entities that amount to the person's body and its constituent parts. A person has the spatial location of its body, which in turn is derived from the spatial location of its constituent physical object. The analogous holds for inanimate entities. By separating the universals that carry different principles of identity, we avoid the problems mentioned by Ríos in (1) above.

The problem mentioned in (2) is solved in figure 8.26 by applying the role modelling design pattern proposed in chapter 4. *Performer Artist* is a mixin universal, since it has as instances individuals that obey incompatible principles of identities, namely, bands (which are kinds of organizations) and individual artists (which are persons). However, in this case the mixin universal *Performer Artist* is a *semi-rigid mixin* (as opposed to a role mixin), since it is a rigid universal for some of its instances (bands) and anti-rigid for others (individual artists).

Figure 8.27 below presents an excerpt of the specification of figure 8.26 that focuses on the alleged derived meronymic relation between a human

heart of a band member and its band. As this specification shows, the relation between a Human Heart and a Band Member is one of *indirect functional parthood* ($i_1$) and the relation between a Band Member and a Band is one of *direct functional parthood* ($d_1$). As discussed in chapter 7, transitivity does not hold across $i_1$ and $d_1$. Thus, in this specific specification, a Human Heart is not part of a Band.

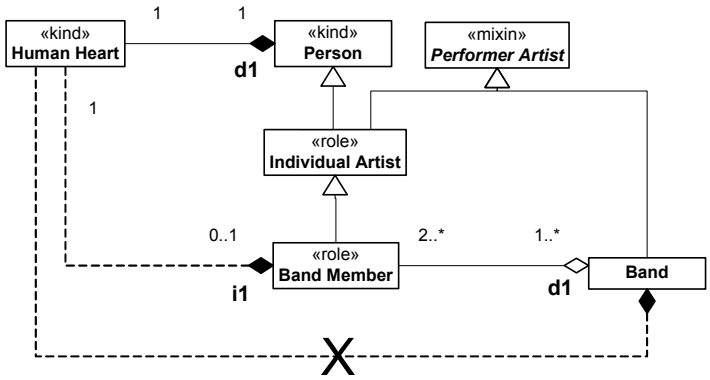



*Figure 8-27* Instantiation of the pattern that exemplifies situations in which transitivity does not hold across functional parthood relations

As an agent, a Person can be a member of a group (figure 8.26). For example, Eric Clapton is a member of the British Guitar Players. However, Clapton's hands are not members of this group. That is, also in this case, transitivity does not hold across the two meronymic relations represented, since the combination of *componentOf* and *memberOf* relations is never transitive (figure 8.28).

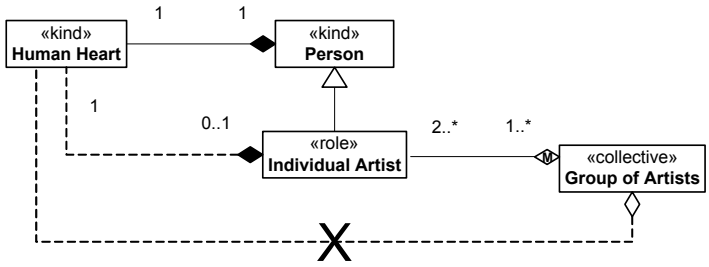



*Figure 8-28* Although a human heart can be part of an individual artist, which in turn is part of a group of artists, a human heart is never a part of a group of artists. This is because the combination of *componentOf* and *memberOf* parthood relations is never transitive

There are a number of material relations in the model of figure 8.26. As it can be noticed, the explicit representation of the foundations of these relations contributes to make their meaning evident. Take for instance the relation *parentOf* between *Parent* and *Offspring*. In this case, we assume that parent is considered in this conceptualization in the legal not in the biological sense, and that in legal terms a person is a parent of another

(offspring) iff the former is registered and legally recognized as such. Therefore, we explicit represent the *Registration* relators that connect parents to their offsprings.

The explicit representation of the relator universal allows for the unambiguous representation of the cardinality constraints associated to the relating universals. This is also the case for the *records* relation between the universals *Performer Artist* and *Track*. The multiplicity one-to-many from Track to Performer Artist leaves open several possible interpretations for the meaning of this relation. Does this multiplicity mean that a track like *Georgia on my mind* can have several recordings (e.g., one by Ray Charles, and another by Jerry Lee Lewis)? By explicitly representing the *Recording* relator, the model makes clear that what is meant by a track is the result of specific recording. However, several artists can participate in one single recording (e.g., both Clapton and B.B.King participate in the recording of *Riding with the King*). Thus, the track *Georgia on my mind* recorded by Ray Charles, and the one recorded by Jerry Lee Lewis are different tracks.

The model of figure 8.22 duplicates the relation creator between artist and album, and artist and track. The intention is to allow for the representation of tracks that are not parts of albums (i.e., that only exist as digital tracks). This situation is modeled in figure 8.26 by having *AlbumTrack* as a restriction of the type Track, in which the *restriction condition* is to be a part of an Album. However, it is unclear in the original model whether these two relations have the same real-world semantics. One interpretation is that, in case a track is part of an album, then the creator of the track must be same as the creator of that album. Still in this interpretation, in the case that different artists participate in the recording of different tracks of the same album (a song collection) they would all be considered creators of that album. The problem is that this interpretation does not allow for the situation in which an artist participates in the recording of one of more tracks of a given album, but is not considered an author of that album. Take, for example, U2's *Rattle & Hum*. Although B.B.King participates in the recording of *Angel of Harlem*, he is not considered an author of that album. We therefore assume that the relation between an artist and an album is one of *legal rights*. This is model in figure 8.26 by an *Authorship* relator universal.

In FOAF, the concept of an agent includes both living and deceased[78] persons. It has also an excessively informal concept of a *Group*, *"covering informal and ad-hoc groups, long-lived communities, organizational groups within a workplace, etc"*[79]. It also allows, for example, for the formation of groups

[78] The FOAF ontology also allows for both real and imaginary characters. We excluded imaginary characters in this specification for the sake of simplicity.
[79] FOAF Vocabulary Specification (http://xmlns.com/foaf/0.1/).

containing only deceased persons. The concept of group in FOAF, hence, seems to include entities that range from collectives with extensional identity principles to social complexes (organizations). We assume here that organizations should be considered as agents, but that mere extensional collections should not. Thus, Agents can be parts of Groups and, in particular, organizations are constituted by Groups. However, Groups are not considered here to be agents.

At first, it seems that the concepts of an agent in FOAF and in MoGATU BDI are equivalent. However, in MoGATU, an agent bears mental states such as beliefs, desires and intentions. We assume here that only living persons and actual organizations can bear these mental states. We use the universal *Active Agent* to model the MoGATU concept of an agent. Active Agent is *mixin* subsuming both *Living Person* and *Active Organization*. Consequently, under these assumptions, a MoGATU agent is an anti-rigid universal. So, for instance, although Aristotle is a (FOAF) Agent, it cannot in the present world have mental states and, consequently, it is not an instance of *Active Agent*. We have an analogous situation for Organizations. For example, *The Beatles* cannot be considered a MoGATU agent in the present world.

Mental states are existentially dependent entities. For example, a belief depends rigidly on a specific bearer active agent, i.e., a particular belief cannot exist without inhering one (and always the same) active agent. Mental states are therefore special types of intrinsic moments (modes) and are represented here as such.

By objectifying intrinsic moments we can also represent their attributes and the relations they participate. For instance, every *intention* has an inhering *priority* quality, which is modeled in figure 8.26 via an attribute function that maps intentions to priority qualia in a *priority level* quality dimension. This quality dimension is a finite set of values ranging from *level 0* to *level 10* and totally ordered under the *higherThan* and *lowerThan* relations.

The relations *moreImportantThan, lessImportantThan* and *asImportantAs* relations between intentions are formal relations derived from the individual priority levels of their relata. The first two relations are anti-symmetric and transitive. The last one is an equivalence relation. As discussed in chapter 6, these meta-properties are derived from the meta-properties of the relations between qualia in the corresponding underlying quality dimension.

## 8.5 Final Considerations

In chapter 2 of this thesis we have proposed a framework for language evaluation and design. This framework establishes a systematic way to evaluate the domain appropriateness of a modelling language by comparing a concrete representation of the universe of discourse referred by the language (represented in terms of a reference ontology) and a meta-model of this language. The goal is to reinforce the strongest possible homomorphism between these two entities.

In this chapter, we provide an exemplification of this framework. As a representation of the selected general (meta-level) domain, we have the foundational cognitive ontology produced throughout chapters 4 to 7 of this thesis. As a selected language, we take the Unified Modeling Language (UML) as a candidate language for conceptual modelling of structural aspects of application domains. The choice for UML is based on the fact that this language is currently a *de facto* standard, and because it represents an interesting case study due to the complexity of its metamodel. Furthermore, there is a growing interest in the adoption of UML as a language for conceptual modelling and ontology representation (Cranefield & Purvis, 1999; Baclawski et al., 2001; Kogut et al., 2002). A more explicit statement of interest on applying UML for this purpose is made by the OMG Ontology Metamodel Definition Request for Proposals (OMG, 2003a): *"The familiarity of users with UML, the availability of UML tools, the existence of many domain models in UML, and the similarity of those models to ontologies suggest that UML could be a means towards more rapid development of ontologies. This approach continues the Object Management Group's 'gradual move to more complete semantic models' as noted in the Model Driven Architecture paper. It would also create a link between the UML community and the emerging Semantic Web community, much as other metamodels and profiles have created links with the developer and middleware communities."*

As demonstrated in sections 8.1 to 8.3 of this chapter, the foundational ontology used suitably supports the redesign of the language's metamodel to obtain a conceptual cleaner, semantically unambiguous and ontologically well-founded version of UML.

Although not discussed here, an extension of this foundational ontology that includes *perdurants* and *intentional and social entities* has been employed to provide a foundation for agent modelling concepts (Guizzardi & Wagner, 2005b), and to analyze the ontological semantics of the some enterprise modeling languages and frameworks (Guizzardi & Wagner, 2005a), among them most notably the REA (Resource-Event-Agent) framework (Geert & McCarthy, 2002). Moreover, a preliminary version of the ontologically well-founded UML produced in this chapter has been used with interesting

results for analyzing conceptual models of genome sequence applications in (Guizzardi & Wagner, 2002).

Finally, in order to show the usefulness of the redesigned version of UML produced, we employ it to tackle some semantically intetoperability problems highlighted by (Ríos, 2003), which can happen in the integration of lightweight ontologies. These problems happen exactly because of the inadequacy of the modelling language used (OWL) in making explicit the underlying ontological commitments of the conceptualizations involved.

The conceptual modelling language proposed here was proven useful in addressing these problems. First, by precisely representing the (modal) meta-properties of the underlying concepts, it allows for an explicit account of their ontological commitments. Second, by providing solutions to classical and recurrent problems in conceptual modelling (e.g., representation of roles with multiple allowed types, the problem of transitivity of parthood relations, the problem of collapsing single-tuple and multiple-tuple multiplicity constraints in the representation of associations, among others), it allows for the production of conceptually clean and semantically unambiguous integrated models.

This case study exemplifies the approach defended in this thesis for semantic interoperation of conceptual models (discussed in chapter 3). We defend that in a first phase of off-line meaning negotiation, an ontologically well-founded modelling language should be used. The main requirements of this language are *domain and comprehensibility appropriateness*. Once this meaning negotiation and semantic interoperation phase is complete, then a knowledge representation language can be used to express the results produced on this phase. The requirements of this second language instead are high computational efficiency and machine-understandability.

In the original scenario proposed by Ríos, the ontologies used are created for the purpose of the example, with the aim to represent stereotypical cases. However, in section 8.4, we demonstrate that real ontologies exist in current Semantic web efforts which are structurally similar to the ones proposed by Ríos, and which are used by practitioners in concrete semantic web applications. Moreover, we also show that there are concrete efforts to unify these separate lightweight ontologies in context-aware applications (e.g., SOUPA) in a manner analogous to the one described in the scenario proposed by Ríos.